Mindset DevOps for QA whit DevSecOps

Programma della giornata

1. Cos’è DevOps
2. Come funziona DevOps
3. Il ciclo di vita DevOps
4. Le 8 fasi DevOps
5. Strumenti DevOps
6. Quali sono i vantaggi DevOps?
7. Quali sono le sfide dell’adozione DevOps?
8. Come adottorare DevOps

Programma della giornata

9. Cos’è DevSecOps

10. Come si implementa DevSecOps

11. Il ruolo QA nel mondo DevOps

12. Responsabilità QA in DevOps

13. Il ruolo strategico del QA in DevOps

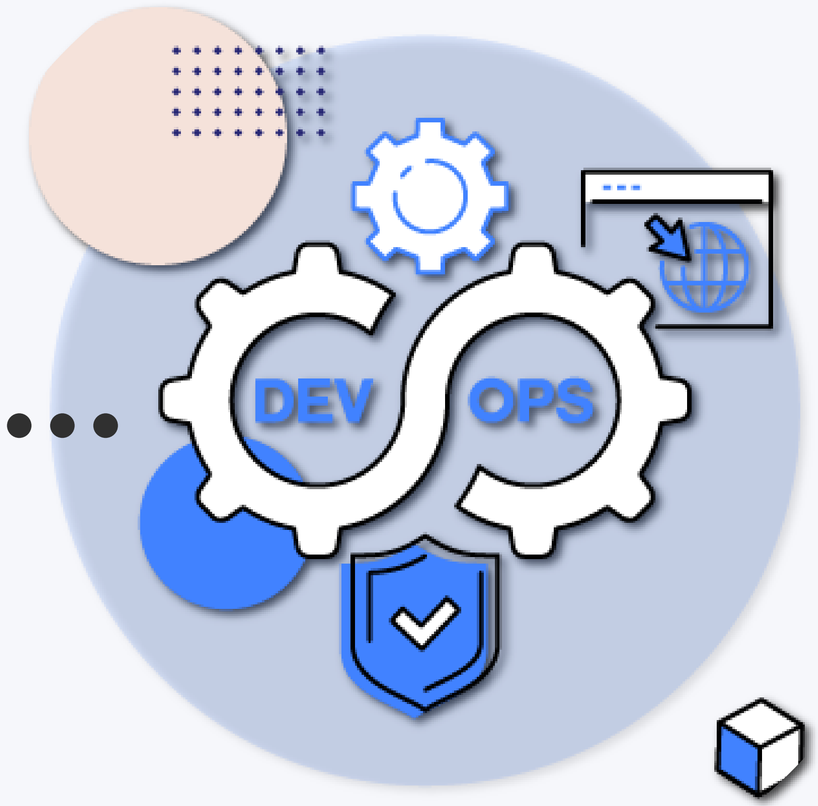
14. 10 best practies QA in uno scenario DevOps

15. Strumenti DevOps: quali sono i migliori tool DevOps da conoscere nel 2023

16. Conclusioni

1 Cos’è DevOps?

Cos’è DevOps?



***Development*:**

Tutte le attività di sviluppo di un prodotto o di un servizio

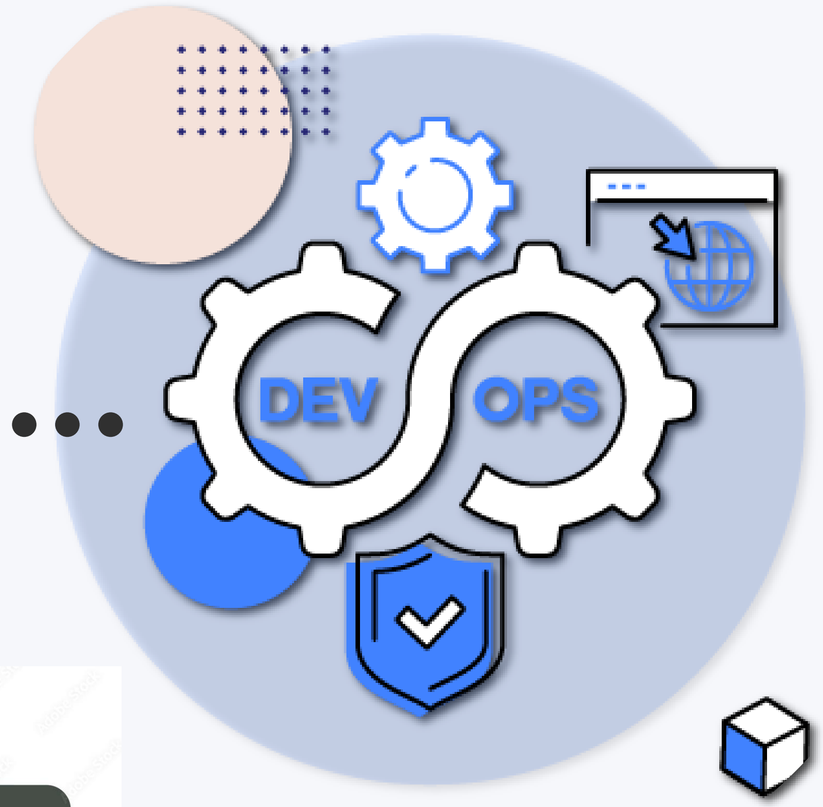
***Operation*:**

Tutte le attività di messa in opera e mantenimento degli stessi

Obiettivo: aumentare la qualità e ridurre il time-to-market

Cos’è DevOps?

DevOps è un insieme di **pratiche**, **strumenti** e una **filosofia culturale** che autorizza ed integra i processi tra i team di sviluppo software e i team IT: in particolare pone l’accento sul potenziamento sulla comunicazione e la collaborazione tra team e sull’automazione tecnologica



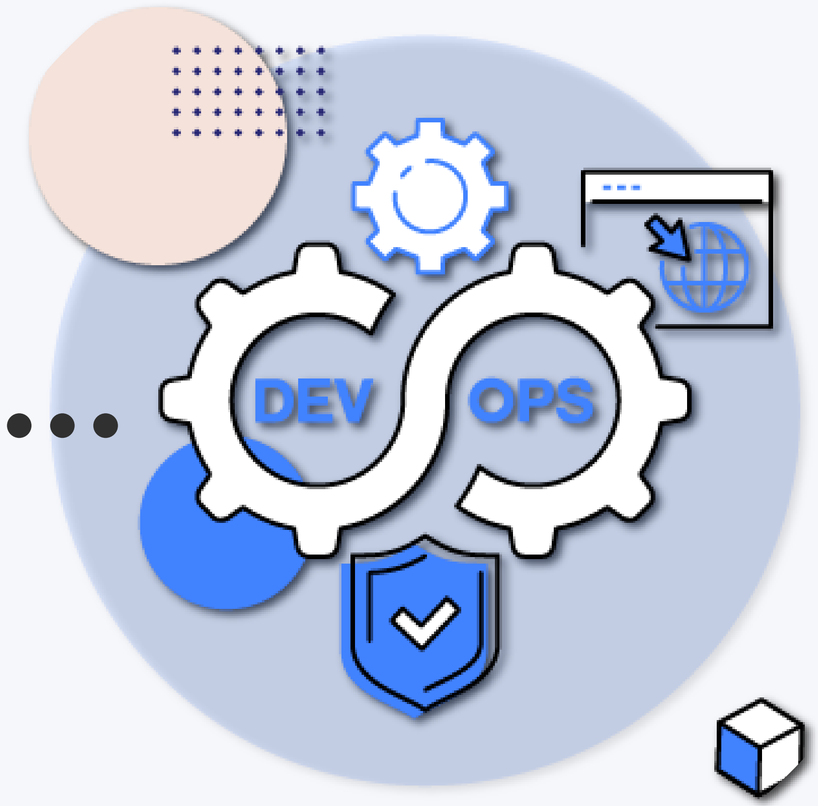
2

Come funziona DevOps

Come funziona DevOps?

Il team DevOps include sviluppatori e addetti alle operazioni IT che collaborano durante l'intero ciclo di vita del prodotto, al fine di aumentare la velocità e la qualità della distribuzione del software.

Nell'ambito del modello DevOps, i team di sviluppo e delle operazioni non sono più "isolati". A volte, questi due team si uniscono in un unico team in cui gli ingegneri lavorano lungo l'intero ciclo di vita dell'applicazione, dallo sviluppo e test fino alla distribuzione e alle operazioni, e dispongono di un'ampia gamma di competenze multidisciplinari.

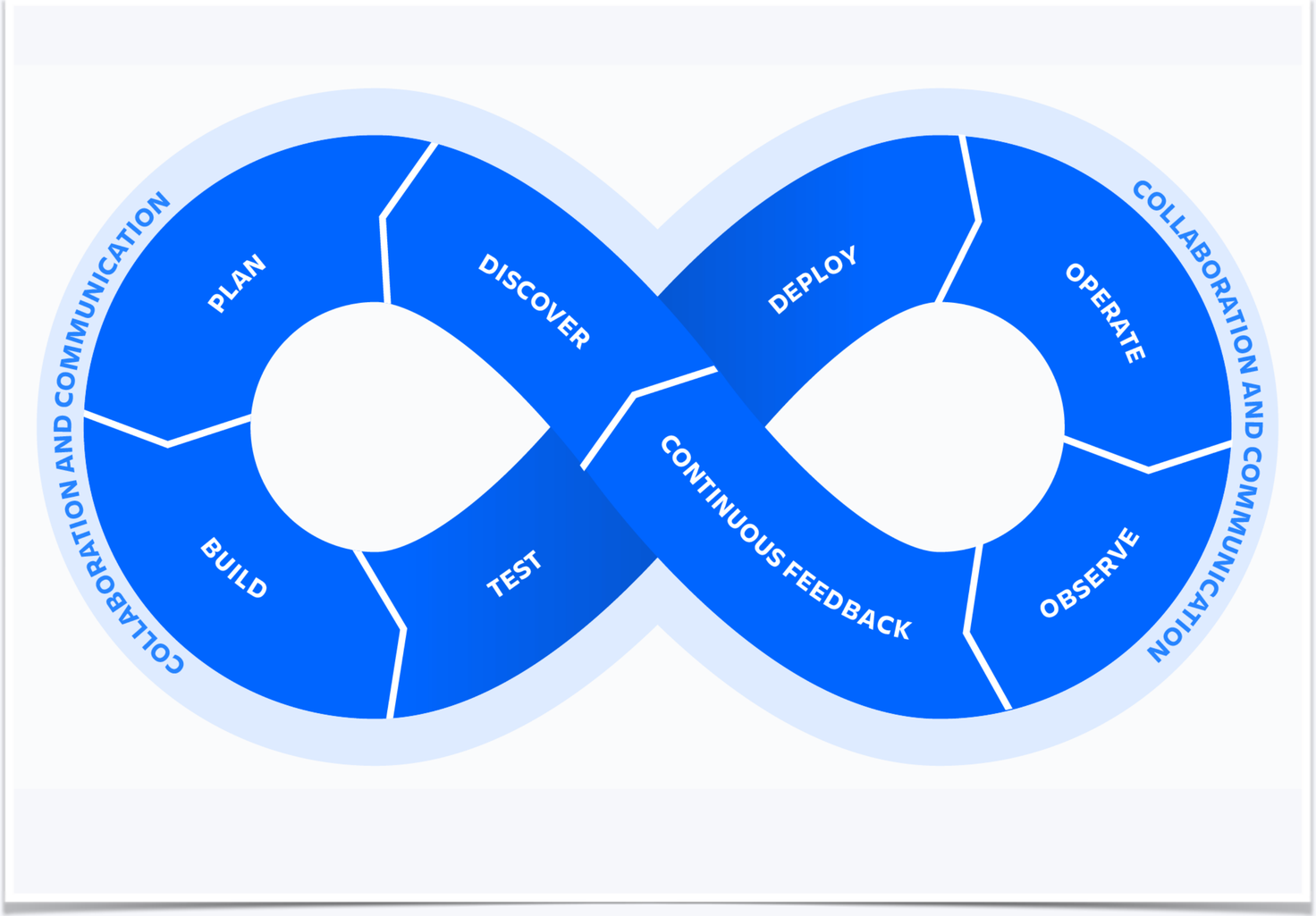


I valori DevOps vengono talvolta applicati a team diversi da quelli di sviluppo. Quando i team di sicurezza adottano l'approccio DevOps, la sicurezza diventa parte attiva e integrata del processo di sviluppo. Questo approccio è denominato DevSecOps.

3 Il ciclo di vita DevOps

Il Ciclo di vita DevOps

A causa della natura continua di DevOps, i professionisti utilizzano il simbolo dell'infinito per mostrare come le fasi del ciclo di vita DevOps si relazionano tra loro.



Nonostante suggerisca l'idea di un flusso continuo, tale simbolo indica la necessità di una collaborazione costante e di un miglioramento iterativo durante l'intero ciclo di vita.

4

Le 8 fasi DevOps

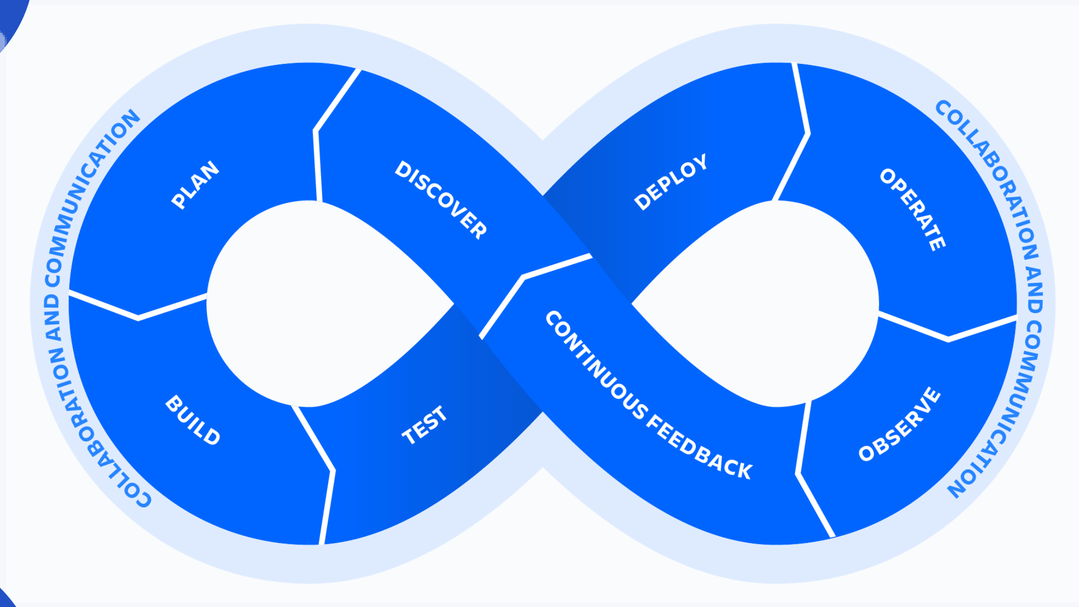
Le 8 fasi DevOps

Scopri

Pianifica

Compila

Testa



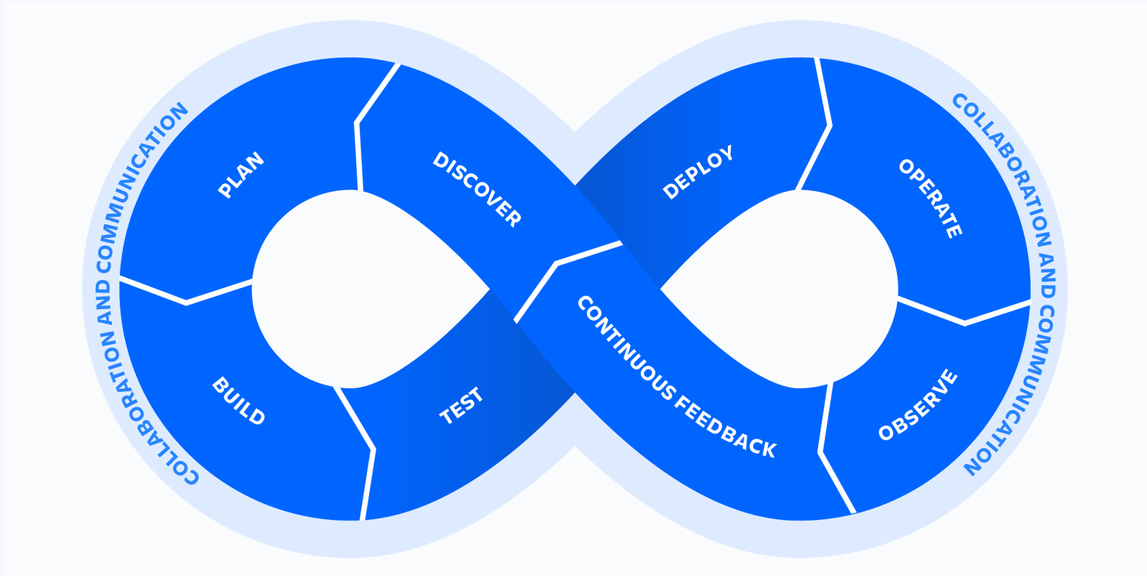
Distribuisci

Esegui Operazioni

Osserva

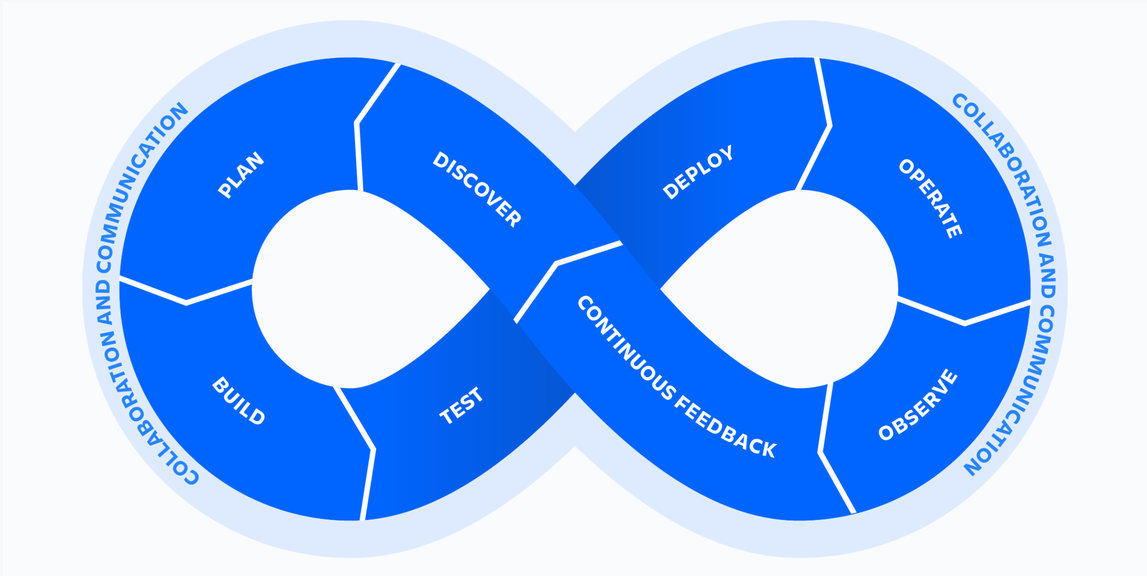
Feedback Continuo

Scopri



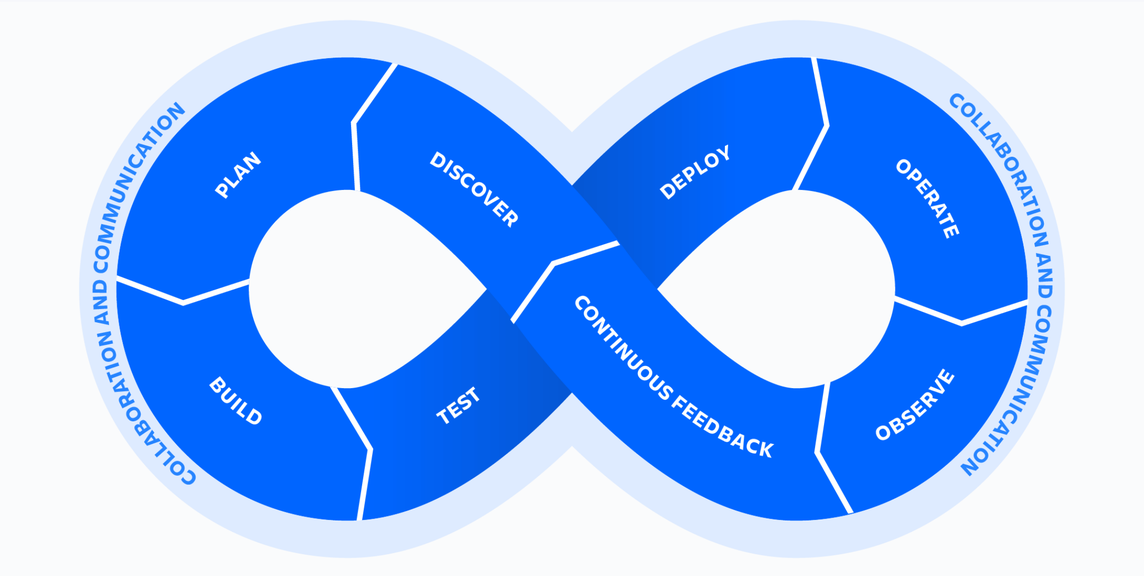
I team devono organizzare workshop per esplorare, organizzare e assegnare la giusta priorità alle idee. Queste ultime, oltre ad allinearsi agli obiettivi strategici, devono avere un impatto sui clienti

Pianifica



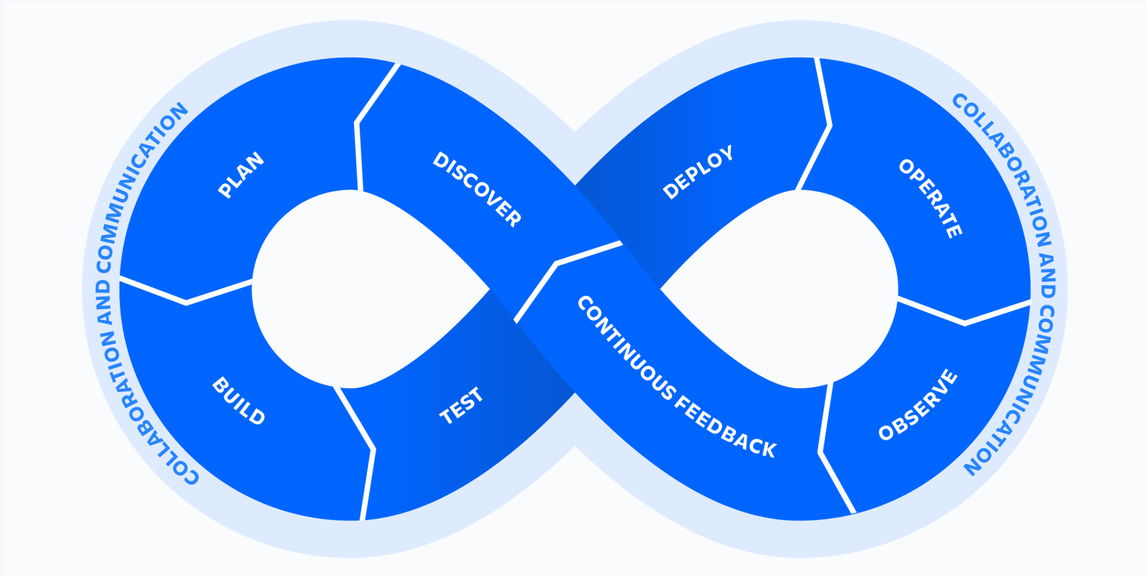
In questa fase la tecnica di co-design pone al centro le esigenze del cliente e si basa sul principio che le persone che utilizzeranno il prodotto sono le più qualificate per contribuire alla sua progettazione e che il loro coinvolgimento può portare a soluzioni più innovative e soddisfacenti

Compila



Nella scrittura del codice applichiamo le best practice di sviluppo del software

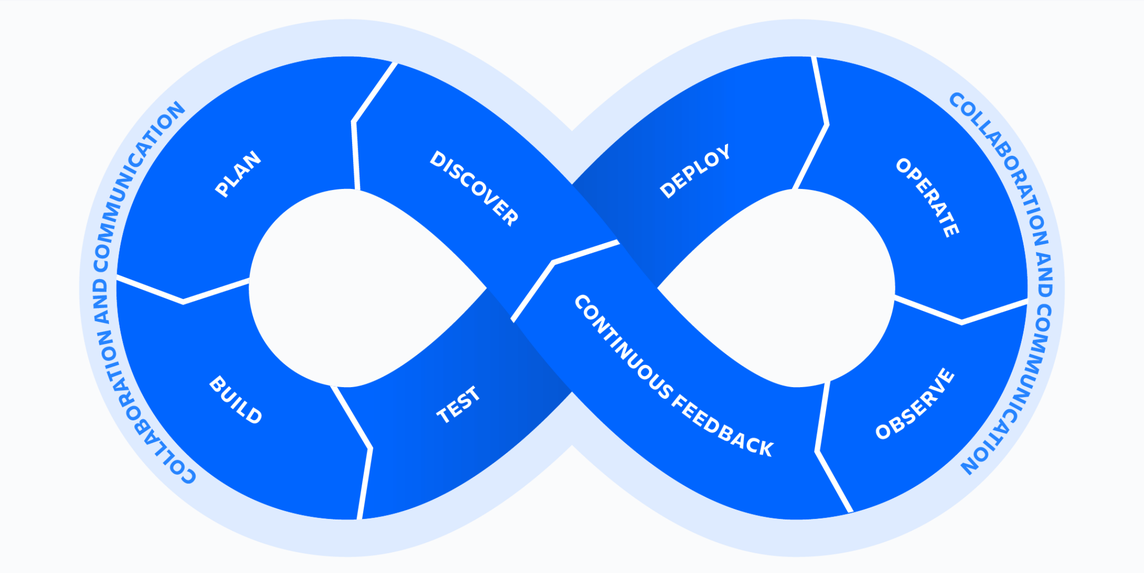
Testa



La fase di testing si compone di feedback continui dagli stakeholder e l’automazione dei test.

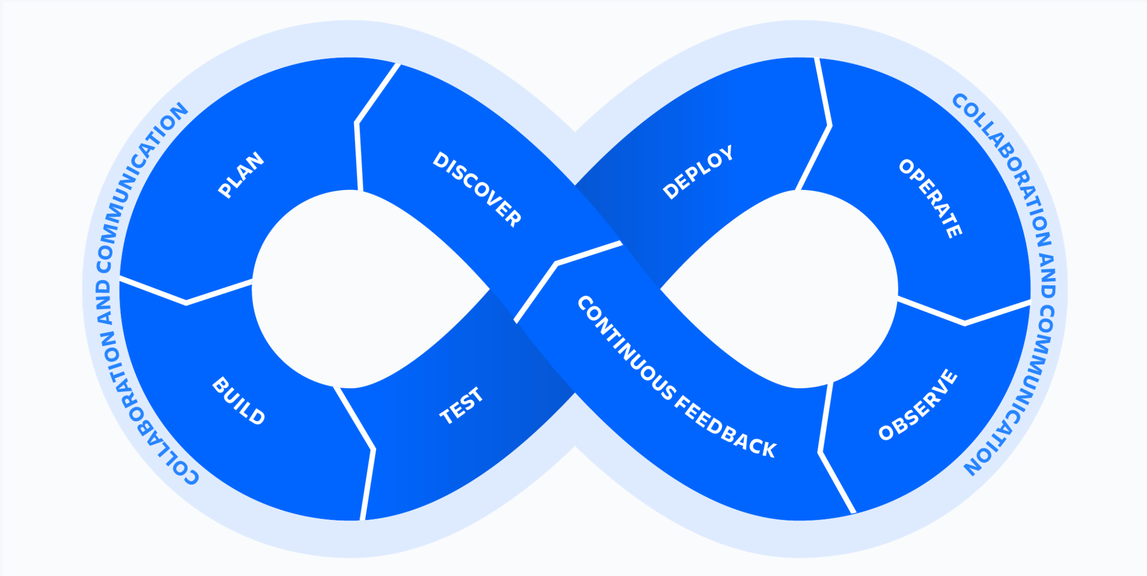
Grazie ai feedback continui, si garantisce l’adeguatezza del codice man mano che viene sviluppato

Distribuisci



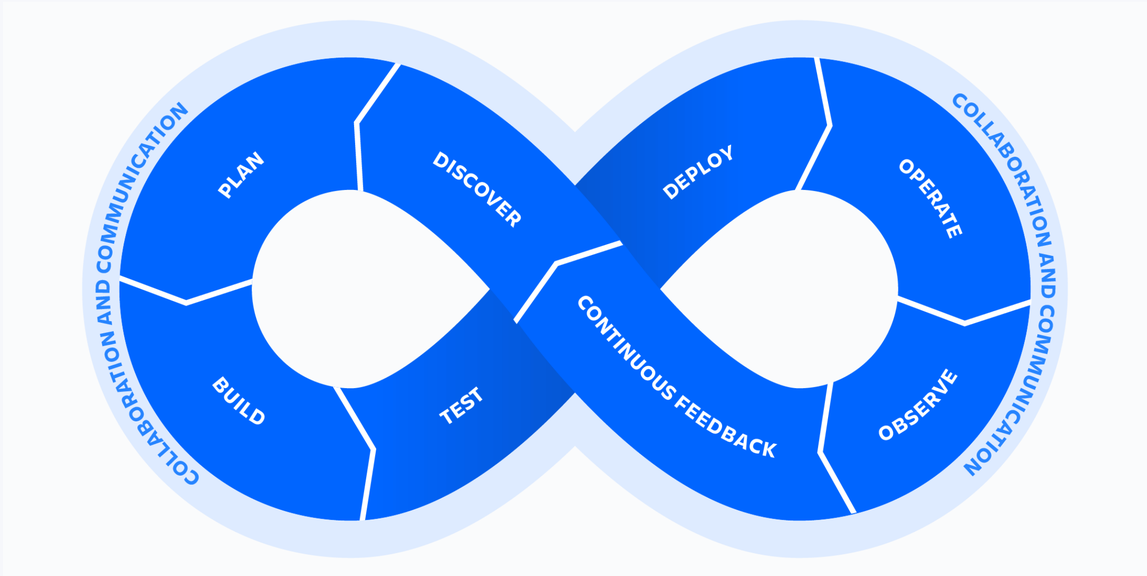
In questa fase sono comprese diverse attività, come la compilazione del codice sorgente, il controllo delle dipendenze, la creazione di pacchetti e le eventuali integrazioni delle modifiche al codice con quello principale. Il risultato finale è un artefatto che può essere rilasciato in un ambiente di produzione

Esegui Operazioni



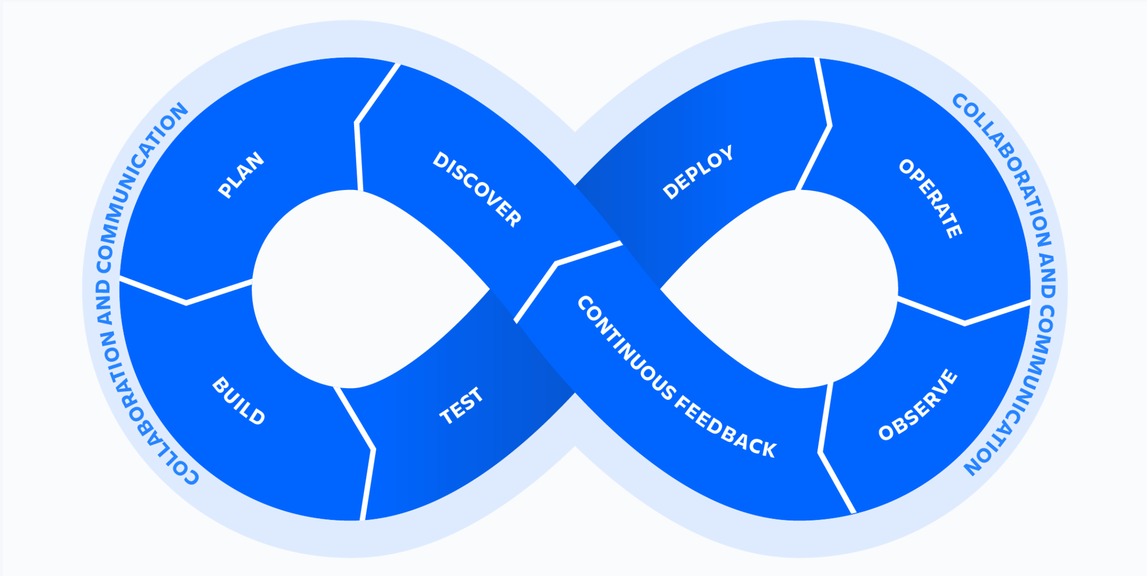
Questa fase è fondamentale nel flusso DevOps, in cui, una volta pronto l’applicativo, si procede con la distribuzione del software al cliente

Osserva



Si Identificano e si risolvono rapidamente i ticket che incidono su tempi di attività, velocità e funzionalità del prodotto. Informa automaticamente il team di modifiche, azioni ad alto rischio o errori, in modo da poter mantenere i servizi attivi

Feedback Continuo



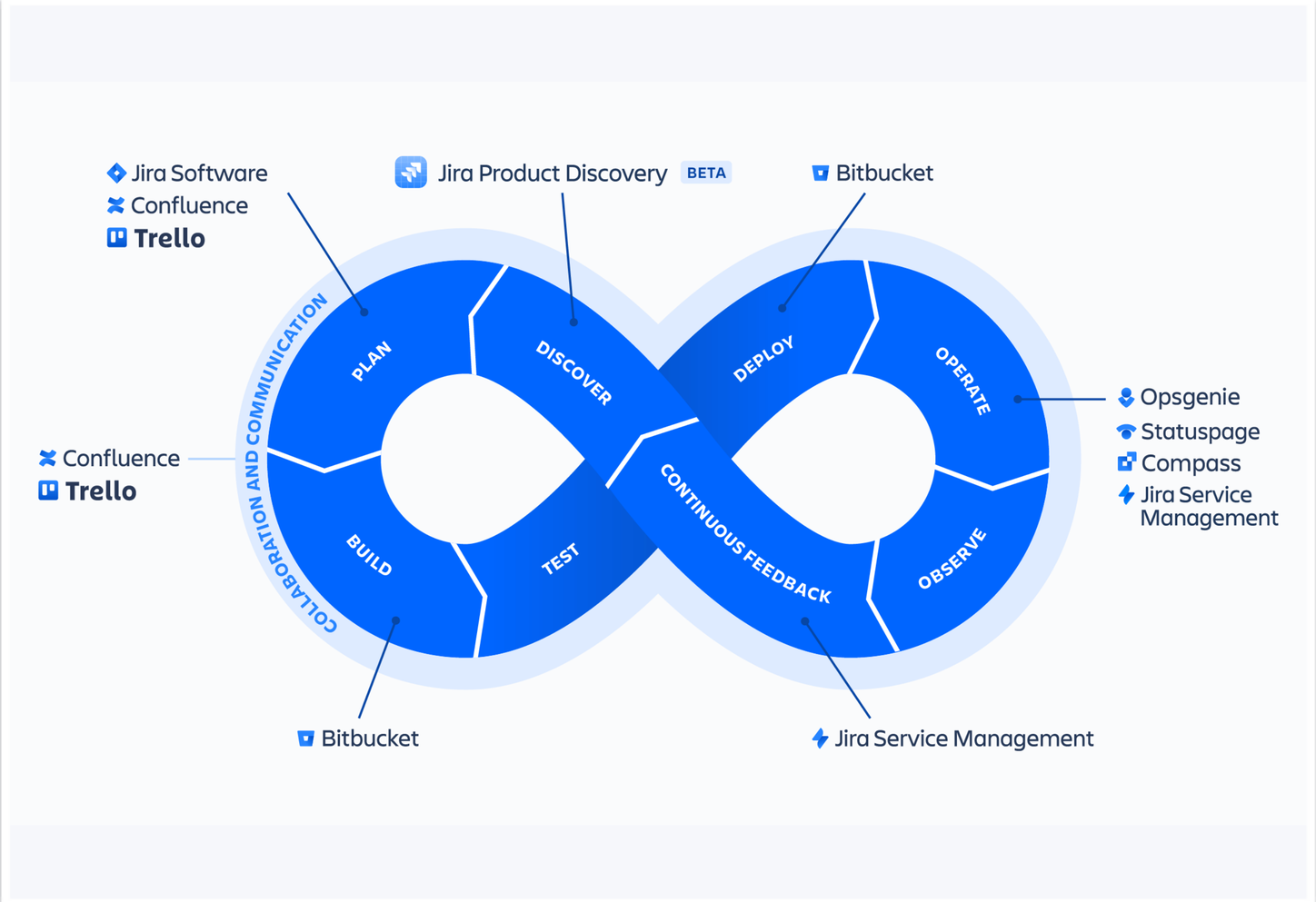
I team DevOps devono valutare ogni rilascio e creare report per migliorare i rilasci futuri.
Raccogliendo feedback continui, i team possono migliorare i processi e incorporare i feedback dei clienti per migliorare i rilasci successivi

5 Strumenti DevOps

Strumenti DevOps

Potenziano le pratiche DevOps contribuendo a:

- **Migliore la collaborazione**
- **Ridurre il cambio di contesto**
- **Introdurre l'automazione**
- **Agevolare l'osservabilità e il monitoraggio**



Strumenti DevOps

Le toolchain DevOps sono in genere di due tipi:

- all-in-one
- aperta

La **toolchain all-in-one** rappresenta una soluzione completa che di solito non si integra con altri strumenti di terze parti

La **toolchain aperta o personalizzabile** consente la personalizzazione con diversi strumenti.

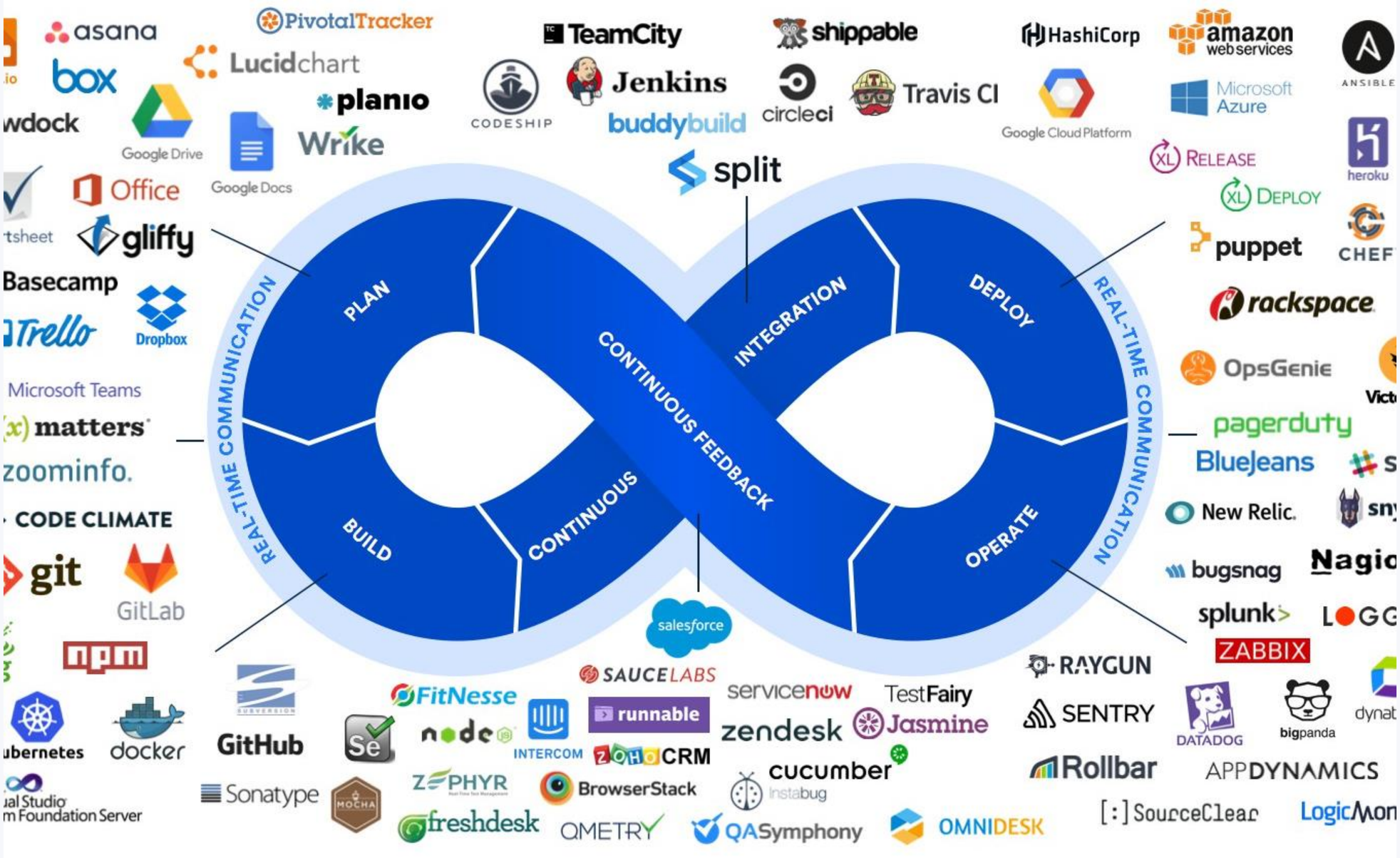
Entrambi gli approcci presentano pro e contro.

Toolchain DevOps

Una toolchain DevOps è una raccolta di strumenti, spesso di fornitori diversi, che operano come unità integrata per progettare, compilare, testare, gestire, misurare e utilizzare software e sistemi.

Consente ai team di sviluppo e operativi di collaborare lungo l'intero ciclo di vita del prodotto e affronta principi fondamentali di DevOps, tra cui la continuous integration, la continuous delivery, l'automazione e la collaborazione.

Toolchain DevOps



Toolchain DevOps

Poiché DevOps rappresenta un cambiamento culturale in cui i team di sviluppo e operativi agiscono come unità integrate, non esiste un unico strumento che abiliti i principi e le pratiche DevOps.

Una toolchain DevOps è invece una raccolta di strumenti, spesso di diversi fornitori, che operano come unità integrata per progettare, compilare, testare, gestire, misurare e utilizzare software e sistemi

Spesso le organizzazioni e i team devono provare diverse combinazioni di strumenti per trovare la toolchain giusta.

Toolchain DevOps

Nella scelta della toolchain DevOps appropriata, è importante innanzitutto comprendere le best practice DevOps fondamentali e il modo in cui gli strumenti le supportano.

Quindi, occorre stabilire una strategia comune per gli strumenti che consenta ai team di collaborare durante lo sviluppo, i test e la distribuzione.

Toolchain All-in-One

Una soluzione DevOps all-in-one è una soluzione completa che potrebbe non integrarsi con altri strumenti di terze parti.

Ciò può essere utile per aziende o gruppi che hanno appena iniziato il loro percorso DevOps o nel caso di un team che desideri avviare rapidamente un progetto

Svantaggi

A. la maggior parte dei team consolidati dispone già di un set di strumenti preferenziali, che potrebbero non integrarsi con una soluzione completa.

B. una toolchain completa potrebbe risentire della sindrome della soluzione generalista: un unico strumento semplicemente non sarà in grado evolversi al ritmo di mercati in rapida evoluzione.

C. il più delle volte le aziende devono integrare strumenti legacy in una toolchain DevOps e una toolchain all-in-one può essere limitante in tal senso.

Toolchain Aperta o Personalizzabile

La toolchain DevOps aperta o personalizzabile può essere personalizzata per le esigenze di un team che utilizza strumenti diversi.

Ciò consente ai team di integrare gli strumenti esistenti che conoscono e apprezzano nella toolchain DevOps più ampia.

Svantaggi

A. L'integrazione è essenziale per questi tipi di toolchain. Se i vari strumenti non si integrano, i membri del team impiegano inutilmente il proprio tempo a passare da una schermata all'altra e ad accedere a più posizioni

B. la condivisione di informazioni tra gli strumenti potrebbe essere difficile.

6

Quali sono i vantaggi DevOps?

Quali sono i vantaggi di DevOps?

I vantaggi di DevOps includono rilasci più rapidi e semplici, efficienza del team, maggiore sicurezza, prodotti di qualità superiore e, di conseguenza, team e clienti più soddisfatti.

Velocità

I team che utilizzano DevOps effettuano rilasci più frequenti, con qualità e stabilità superiori.

La continuous de ente a
di creare, testa e sc
avvalendosi di nati

Sicurezza

Integrando la sicurezza in una pipeline di continuous integration, continuous delivery e continuous deployment, **DevSecOps** è parte attiva e integrata del processo di sviluppo.

La sicurezza è incorporata nel prodotto tramite l'integrazione di controlli di protezione attivi e test di sicurezza nei flussi di lavoro

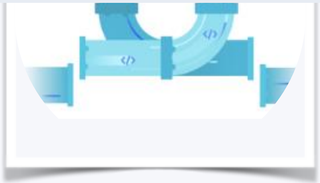
Aumentando la frequenza e la velocity dei rilasci, i team DevOps migliorano rapidamente i prodotti.

apidi di nuove funzioni e le
correzioni dei bug consentono
un vantaggio competitivo.

Collaborazione Migliorata

DevOps è una cultura di collabor
tra i team di sviluppatori e quel
operazioni, che condividono le responsabilità e collaborano tra di loro.

Ciò li rende più efficienti e consente di risparmiare tempo



Qualità e Affidabilità

come continuous integration e
delivery assicurano che le modifiche siano funzionali e sicure, con conseguente miglioramento della qualità dei prodotti software.

Il monitoraggio aiuta i team a restare informati sulle prestazioni in tempo reale.

7

Quali sono le sfide dell’adozione DevOps?

Quali sono le sfide dell’adozione di DevOps?

È difficile rompere le abitudini. Alcuni team potrebbero erroneamente credere che i nuovi strumenti siano sufficienti per l'implementazione di DevOps.

Dimenticano però che DevOps è una combinazione di persone, strumenti e cultura.

Tutti i membri di un team DevOps devono conoscere l'intero flusso di valore, dall'ideazione, allo sviluppo, fino all'esperienza dell'utente finale.

Quali sono le sfide dell’adozione di DevOps?

ATLASSIAN

...

> DevOps non è il lavoro di una singola persona. È il lavoro di tutti.
>
> Robert Krohn
>
> HEAD OF ENGINEERING, DEVOPS DI ATLASSIAN

Atlassian Corporation Plc è una compagnia di software enterprise Australiana che sviluppa prodotti per sviluppatori di software, project management e per la gestione dei contenuti.

Atlassian possiede oltre 135.000 clienti.

[info da Wikipedia]

8 Come adottare DevOps

Come adottare DevOps

L'adozione di DevOps richiede innanzitutto l'impegno a valutare ed eventualmente modificare o rimuovere team, strumenti o processi attualmente in uso nell'organizzazione.

Ciò significa creare l'infrastruttura necessaria per dare ai team l'autonomia di cui hanno bisogno per compilare, distribuire e gestire i loro prodotti senza dover fare troppo affidamento sui team esterni.

Cultura DevOps

Nell'ambito della cultura DevOps i team adottano nuove modalità di lavoro che implicano collaborazione e comunicazione maggiori.

È un allineamento di persone, processi e strumenti verso un focus più unificato in direzione del cliente. I team multidisciplinari si assumono la responsabilità dell'intero ciclo di vita dei prodotti.

Apprendimento Continuo

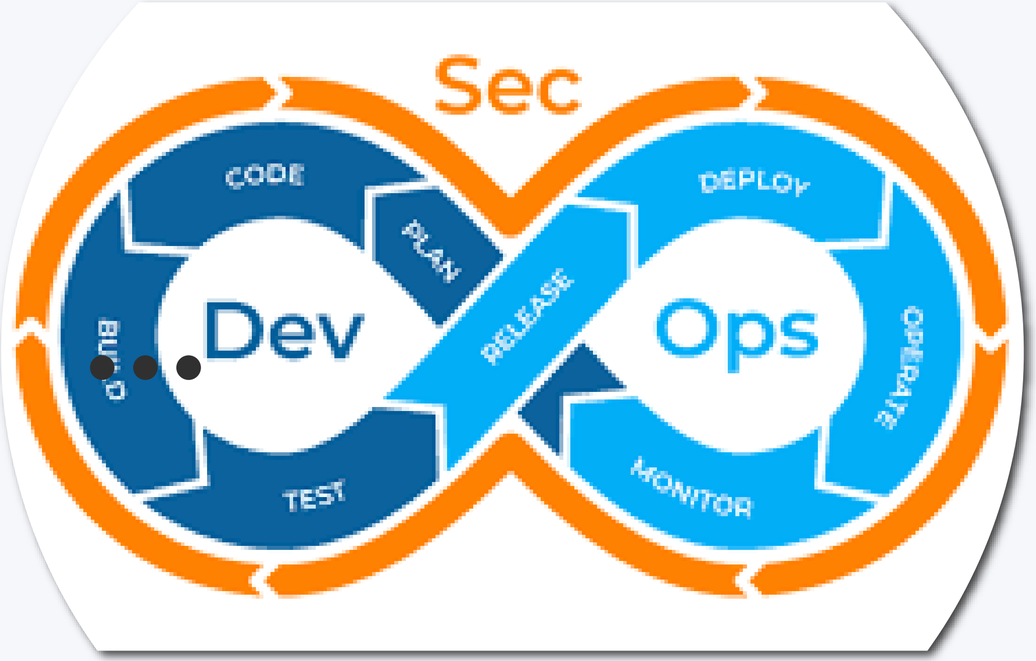
Nelle organizzazioni che implementano correttamente DevOps, vengono incoraggiate la sperimentazione e l'assunzione di rischi.

In questo tipo di organizzazioni, pensare fuori dagli schemi è la norma e i fallimenti sono visti come un aspetto naturale dell'apprendimento e del miglioramento.

9

Cos’è DevSecOps?

Cos’è DevSecOps?

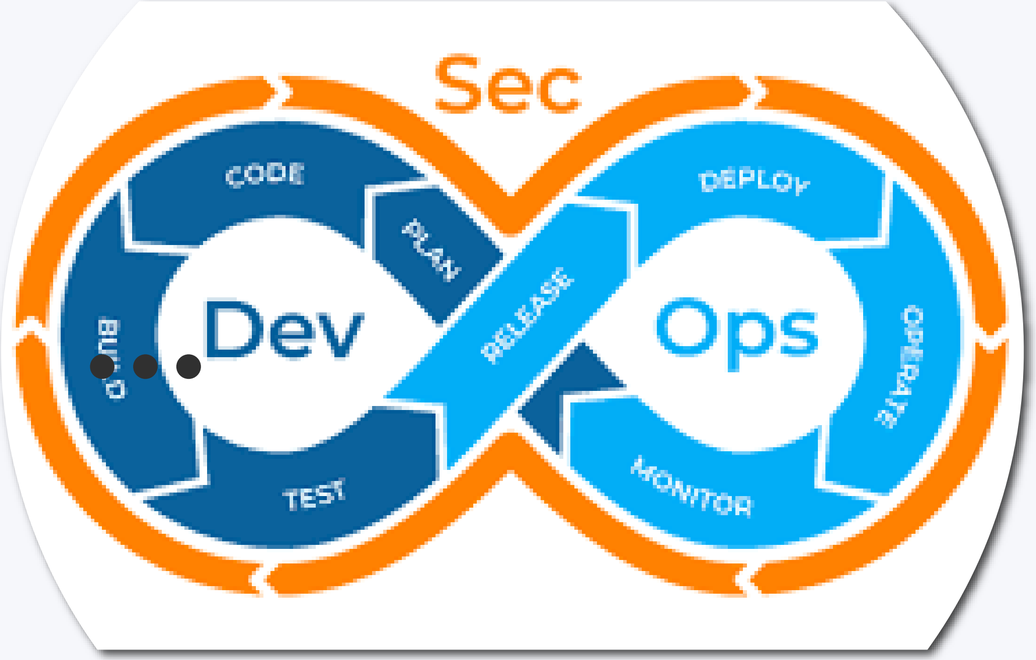


Nonostante le lodevoli iniziative delle aziende di software, le violazioni della sicurezza continuano a verificarsi.

In parte il problema risiede nel fatto che man mano che aumentano la scalabilità e la complessità delle applicazioni software aumenta anche la superficie esposta a vulnerabilità ed exploit della sicurezza.

Dal 2000, circa 3,5 miliardi di persone hanno subito il furto dei dati personali

Cos’è DevSecOps?

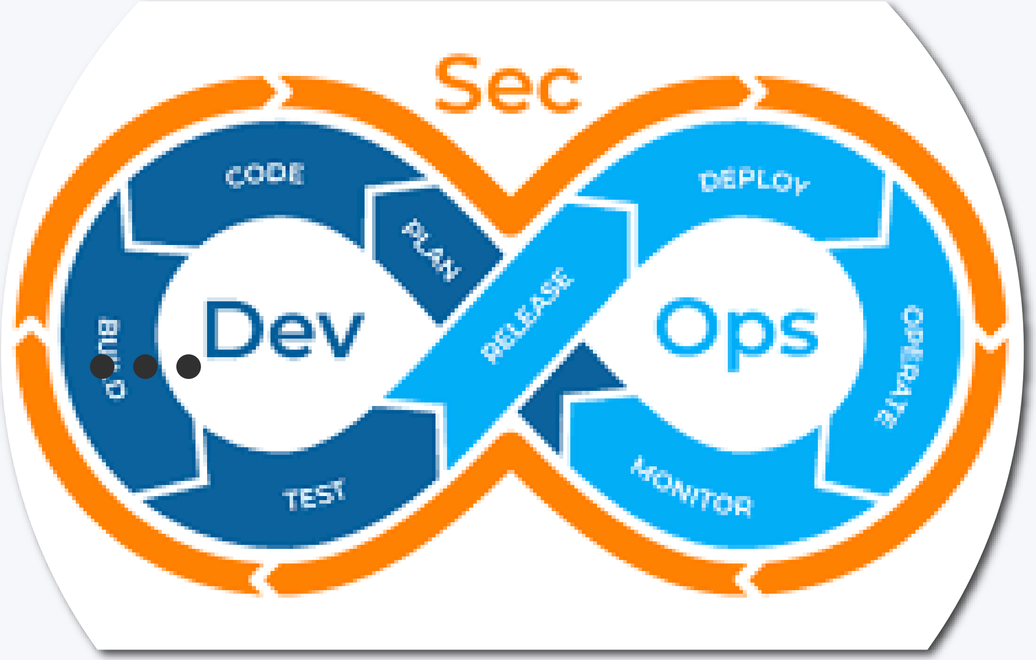


Sempre più organizzazioni adottano un approccio DevOps, che automatizza e integra i processi tra i team di sviluppo software e IT

gli strumenti di sicurezza tradizionali spesso non sono più adeguati. Oggi gli sviluppatori devono integrare misure di sicurezza in ogni fase del flusso di lavoro di sviluppo.

Quando è applicata alla sicurezza dei flussi di lavoro DevOps, questa pratica viene definita DevSecOps

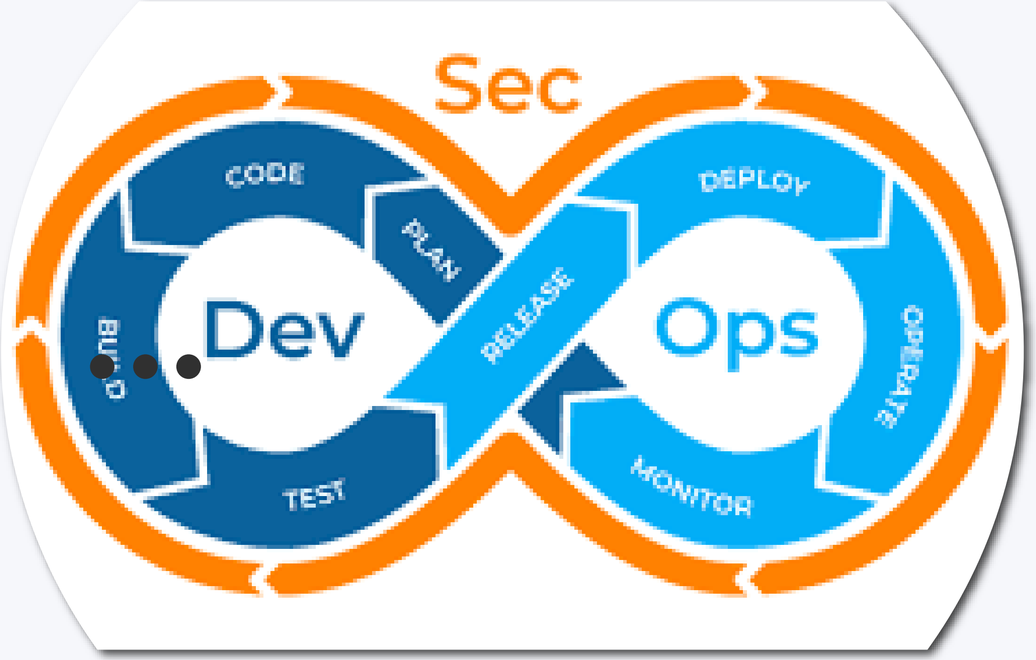
Cos’è DevSecOps?



DevSecOps indica la pratica di integrare la sicurezza in una pipeline di continuous integration, continuous delivery e continuous deployment.

Incorporando i valori DevOps nella sicurezza software, la verifica della sicurezza diventa una parte attiva e integrata del processo di sviluppo.

Cos’è DevSecOps?



Proprio come DevOps, DevSecOps è una metodologia organizzativa e tecnica che combina flussi di lavoro di gestione dei progetti con strumenti IT automatizzati.

DevSecOps integra controlli di sicurezza attivi e test di sicurezza nello sviluppo e nei flussi di lavoro DevOps in modo che la **sicurezza sia integrata nel prodotto** invece di essere applicata a un prodotto finito.

10

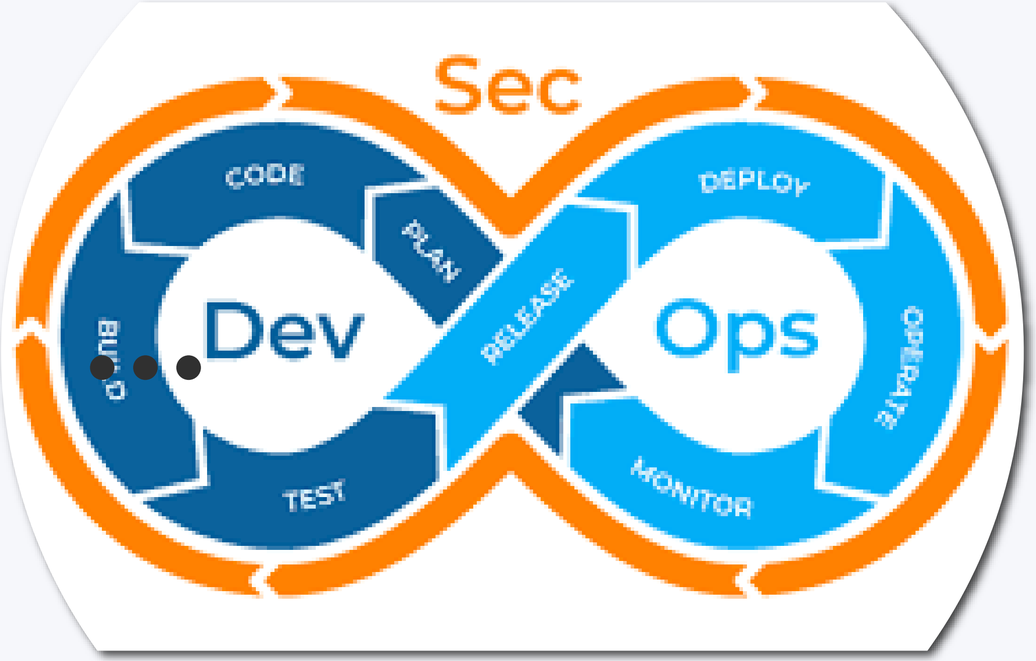
Come si implementa DevSecOps?

Come si implementa DevSecOps?

A. Si introduce la sicurezza nel corso dell'intero ciclo di vita dello sviluppo software per ridurre al minimo le vulnerabilità nel codice software.

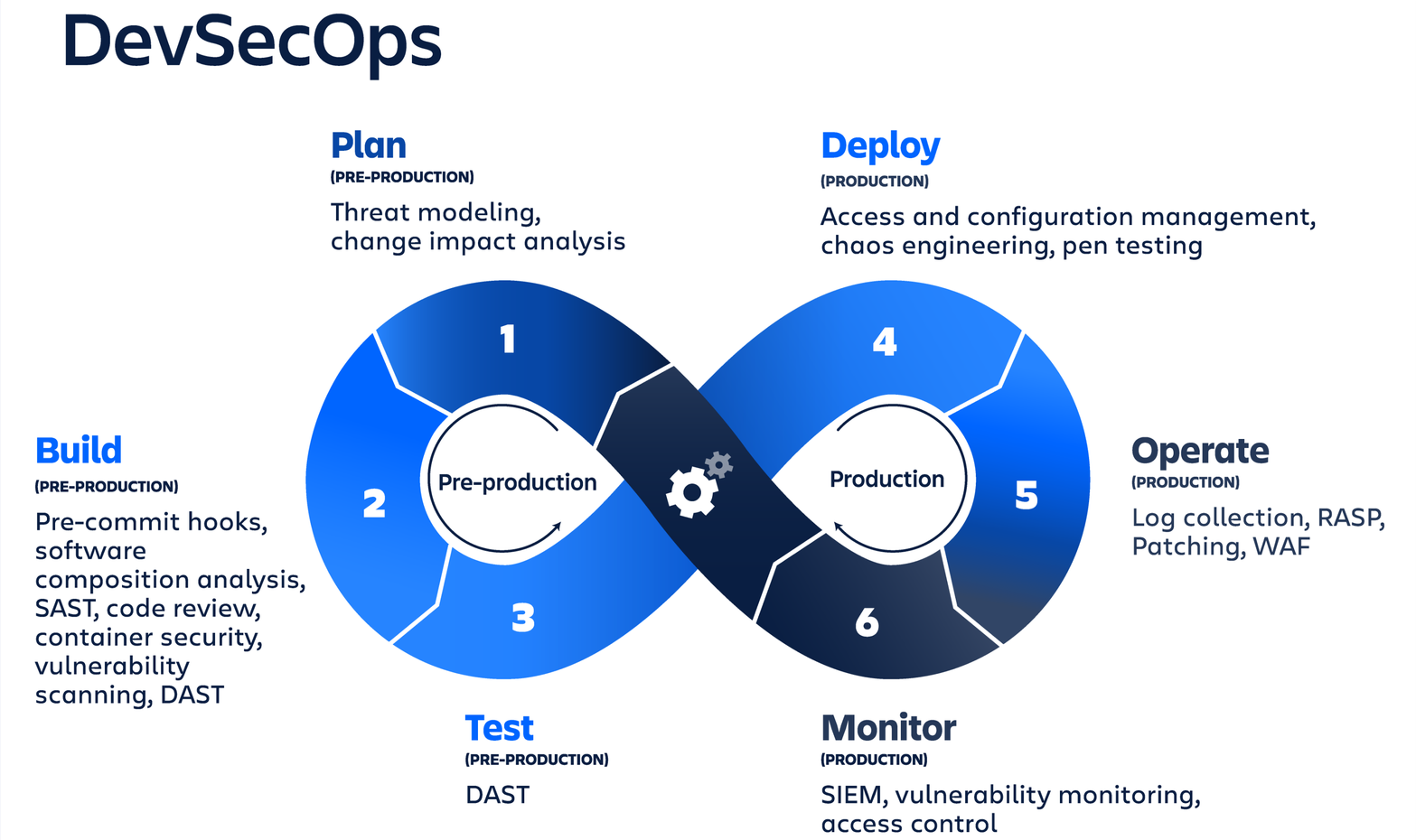
B. Si garantisce che l'intero team DevOps, incluso il team di sviluppo e quello operativo, condivida la responsabilità di seguire le best practice di sicurezza.

C. Si pongono in essere controlli di sicurezza automatizzati in ogni fase della distribuzione del software tramite l'integrazione di controlli, strumenti e processi di sicurezza nel flusso di lavoro DevOps.



DevSecOps

Con DevSecOps, la sicurezza dovrebbe essere applicata a ogni fase della tipica pipeline DevOps: pianificazione, build, test, distribuzione, funzionamento e monitoraggio.



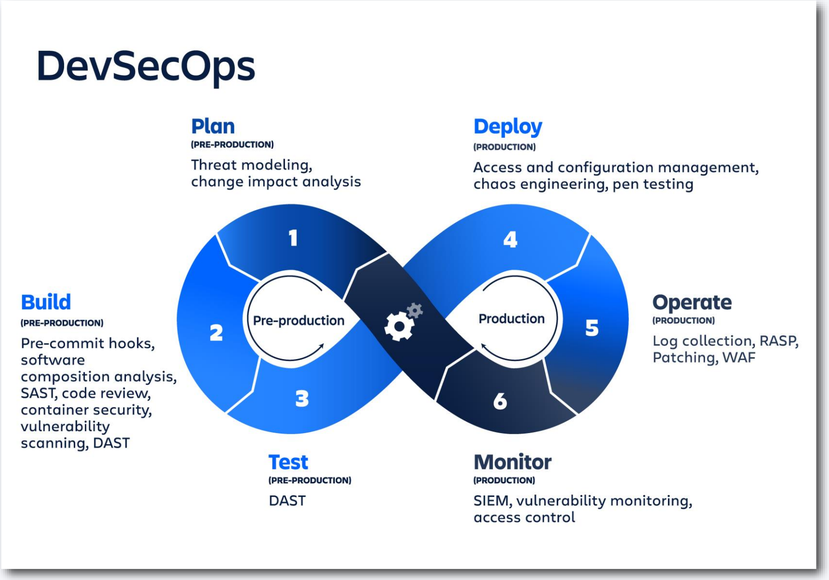
DevSecOps - Pianificazione

La **Pianificazione** è la fase meno automatizzata di DevSecOps perché prevede attività di collaborazione, discussione, revisione e strategia di analisi della sicurezza.

I team devono eseguire un'analisi di sicurezza e creare un piano che delinei dove, come e quando verranno eseguiti i test di sicurezza.

Uno strumento di pianificazione molto diffuso per DevSecOps è **IriusRisk**, uno strumento di progettazione collaborativa per la modellazione delle minacce.

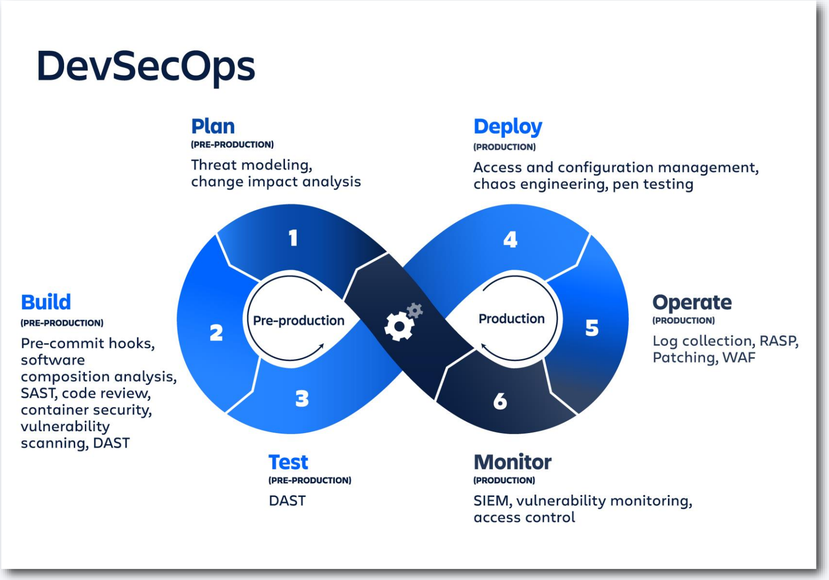
Altri strumenti includono strumenti di monitoraggio e gestione dei ticket come Jira Software e strumenti di comunicazione e chat come Slack.



DevSecOps - Compilazione

La fase di **build** inizia quando gli sviluppatori eseguono il commit del codice nel repository di origine.

Gli strumenti di build di DevSecOps si concentrano sull'analisi automatica della sicurezza rispetto all'artefatto dell'output della build. Importanti pratiche di sicurezza includono l'analisi dei componenti software, i test di sicurezza delle applicazioni statiche (SAST) e i test unitari.



Gli sviluppatori installano e compilano regolarmente utilizzando dipendenze di codice di terze parti, che possono provenire da una fonte sconosciuta o non attendibile.

Le dipendenze del codice esterno possono includere, accidentalmente o a causa dell’azione di utenti malevoli, vulnerabilità ed exploit.

Durante la fase di build, è fondamentale esaminare e analizzare queste dipendenze per cercare eventuali vulnerabilità della sicurezza.

Alcuni strumenti ben noti per eseguire l'analisi della fase di build includono: OWASP Dependency-Check, SonarQube, SourceClear, Retire.js, Checkmarx e Snyk.

DevSecOps - Test

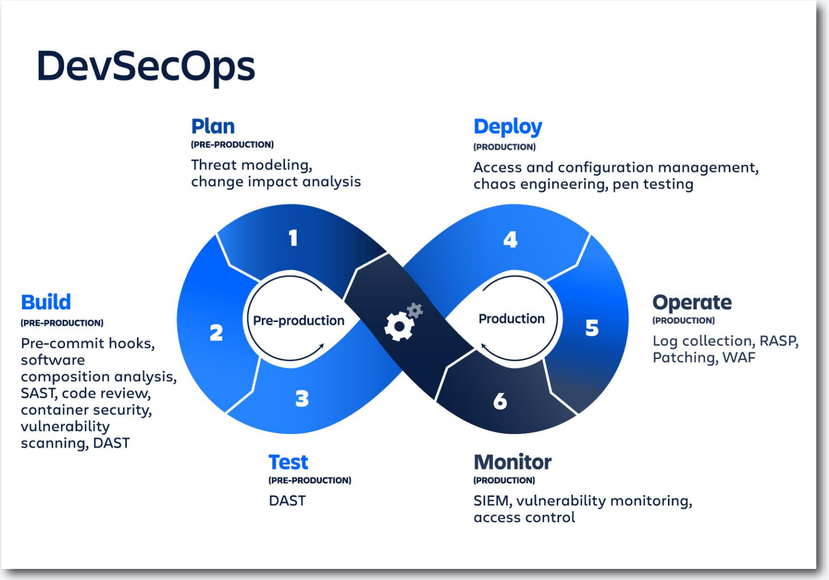
La fase di **test** viene attivata dopo che un artefatto di build è stato creato e distribuito correttamente in ambienti di staging o test. L'esecuzione di una suite di test completa richiede molto tempo.

Questa fase dovrebbe essere sottoposta a fail-fast in modo che le attività di test più costose siano lasciate alla fine.

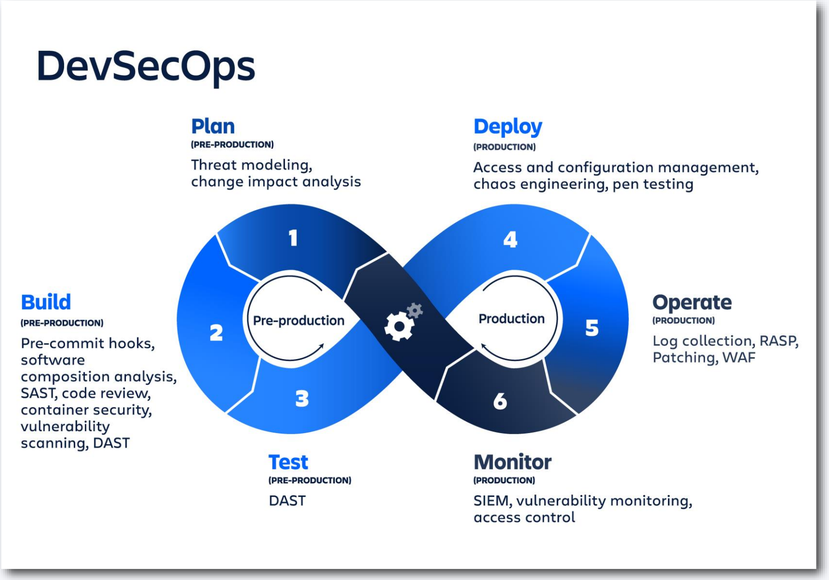
La fase di test utilizza strumenti di test della sicurezza delle applicazioni dinamiche (DAST) per rilevare flussi di applicazioni live come autenticazione utente, autorizzazione, SQL Injection ed endpoint correlati alle API.

I test DAST incentrati sulla sicurezza analizzano un'applicazione rispetto a un elenco di problemi noti di gravità elevata, come quelli elencati nell'**OWASP Top 10**.

Sono disponibili numerosi strumenti di test open source e a pagamento, che offrono una varietà di funzionalità e supporto per gli ecosistemi linguistici, tra cui BDD Automated Security Tests, JBroFuzz, Boofuzz, OWASP ZAP, Arachni, IBM AppScan, GAUNTLT e SecApps Suite.



DevSecOps - Distribuzione



Se le fasi precedenti vengono superate correttamente, è possibile distribuire in produzione l'artefatto della build.

La fase di **distribuzione** è quella ideale per utilizzare strumenti di verifica runtime come Oquery, Falco e Tripwire, che estraggono informazioni da un sistema in esecuzione per stabilire se il funzionamento è quello previsto.

Netflix è ampiamente noto per il suo **strumento Chaos Monkey**, che applica i principi dell'ingegneria del caos. Netflix utilizza anche lo strumento **Security Monkey** che cerca violazioni o vulnerabilità in gruppi di sicurezza dell'infrastruttura configurati in modo errato ed elimina gli eventuali server vulnerabili.

DevSecOps - Distribuzione

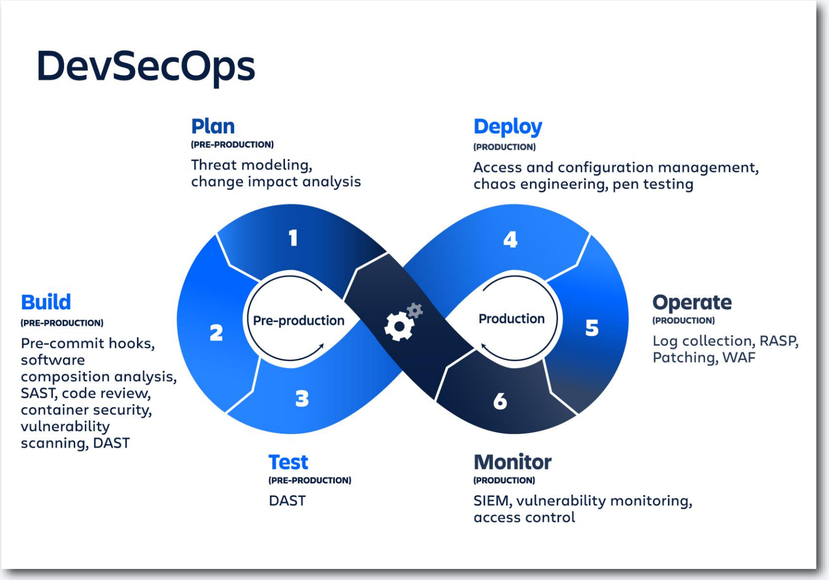
Il **principio del privilegio minimo (PoLP)** è una delle principali preoccupazioni della fase di rilascio.

PoLP indica che qualsiasi utente, programma o processo dispone di un privilegio di accesso minimo per svolgere la sua funzione. Ciò comporta il controllo delle chiavi API e dei token di accesso in modo che i responsabili dispongano di accesso limitato.

In assenza di questo controllo, un utente malintenzionato potrebbe trovare una chiave per accedere ad aree riservate del sistema.

Gli strumenti di **gestione della configurazione** rappresentano un ingrediente essenziale per la sicurezza nella fase di rilascio, poiché forniscono visibilità sulla configurazione statica di un'infrastruttura dinamica.

La configurazione del sistema può quindi essere verificata e rivista: la configurazione diventa immutabile e può essere aggiornata solo tramite commit in un repository di gestione della configurazione.



Alcuni strumenti di gestione della configurazione di ampia diffusione includono Ansible, Puppet, HashiCorp Terraform, Chef e Docker.

DevSecOps - Osservazione

Dopo che un'applicazione viene distribuita e stabilizzata in un ambiente di produzione live, sono necessarie ulteriori misure di sicurezza.

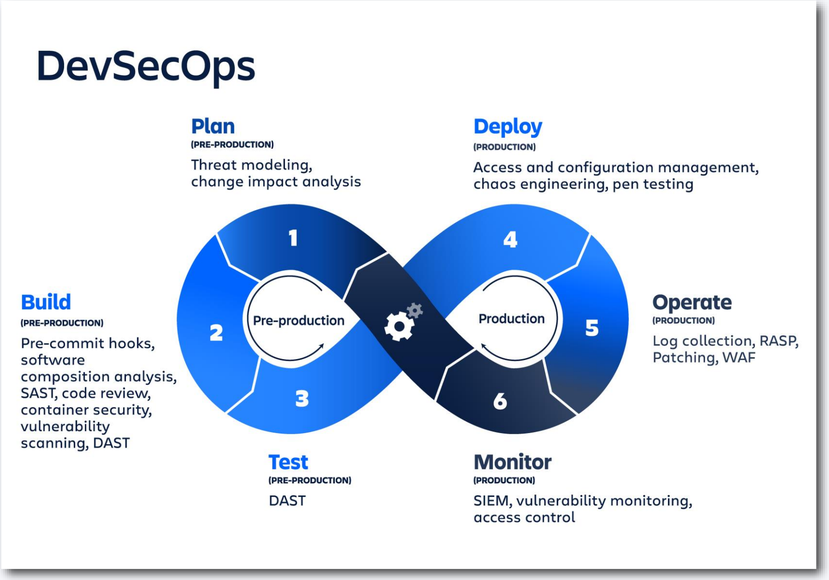
Le aziende devono **monitorare** e osservare l'applicazione live per rilevare eventuali attacchi o perdite con controlli di sicurezza automatizzati e cicli di monitoraggio della sicurezza.

Runtime Application Self-Protection (RASP) identifica e blocca automaticamente le minacce alla sicurezza in entrata in tempo reale.

RASP funge da proxy inverso che osserva gli attacchi imminenti e consente all'applicazione di riconfigurarsi automaticamente senza l'intervento umano in risposta a condizioni esplicite.

Un team interno o esterno specializzato può eseguire test di penetrazione per individuare exploit o vulnerabilità attraverso la compromissione intenzionale di un sistema.

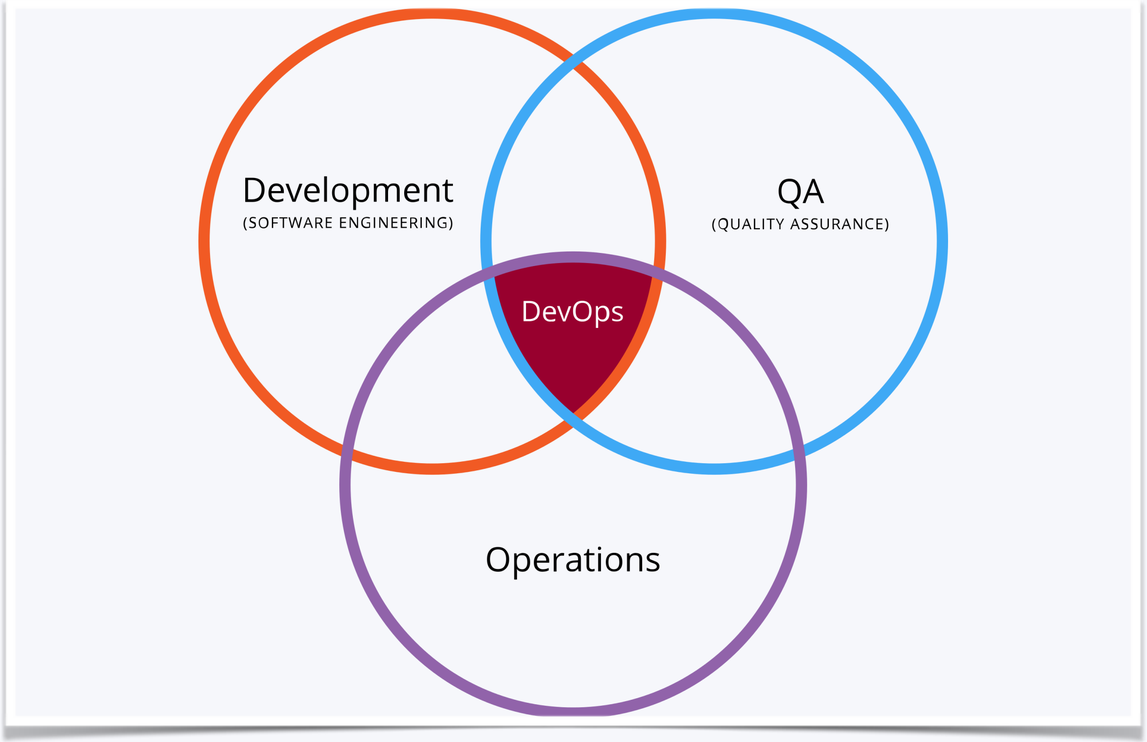
Un'altra tecnica di sicurezza è l'utilizzo di un programma Bug Bounty che offre una ricompensa a persone esterne che segnalano exploit e vulnerabilità della sicurezza.
Alcuni esempi di strumenti di difesa runtime più diffusi includono Imperva RASP, Alert Logic e Halo.



11

Qual’è il ruolo del QA nel mondo DevOps?

Qual’è il ruolo del QA in DevOps?



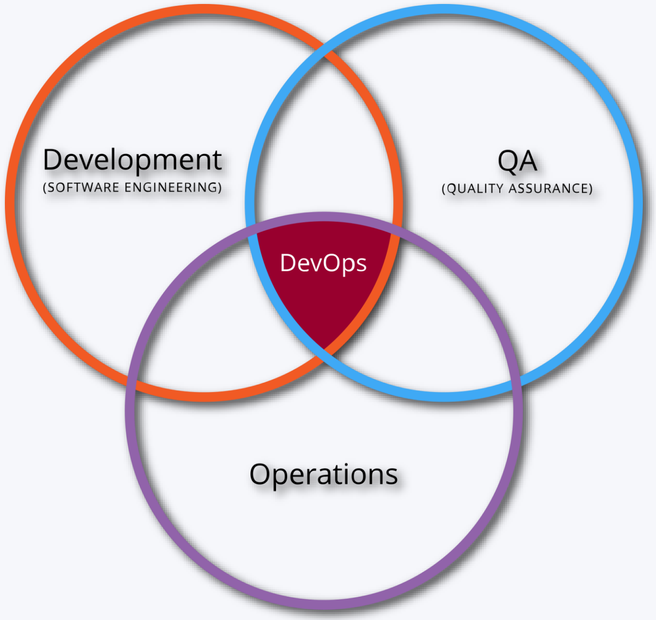
In precedenza, le organizzazioni di sviluppo software funzionavano attraverso due entità autonome, vale a dire sviluppatori e QA.

Entrambi, compresi i loro ruoli e le loro aspettative, sono considerati attività separate svolte al fine di fornire un prodotto. Tuttavia, questo esclude le persone che non appartengono al processo di ingegneria (ad es. Operazioni), mentre lo sviluppo e il QA lavorano in modo coeso e unanime per raggiungere un obiettivo comune.

Qual’è il ruolo del QA in DevOps?

Comunemente il QA ottiene la build una volta che viene distribuita in un ambiente designato e segue l'esecuzione del processo di test

Quando la build è stata verificata da QA, va in produzione. Questo processo di solito richiede un paio di giorni nell'ambiente QA (**se ci dice bene!!)**, ma questo non è il caso di DevOps.



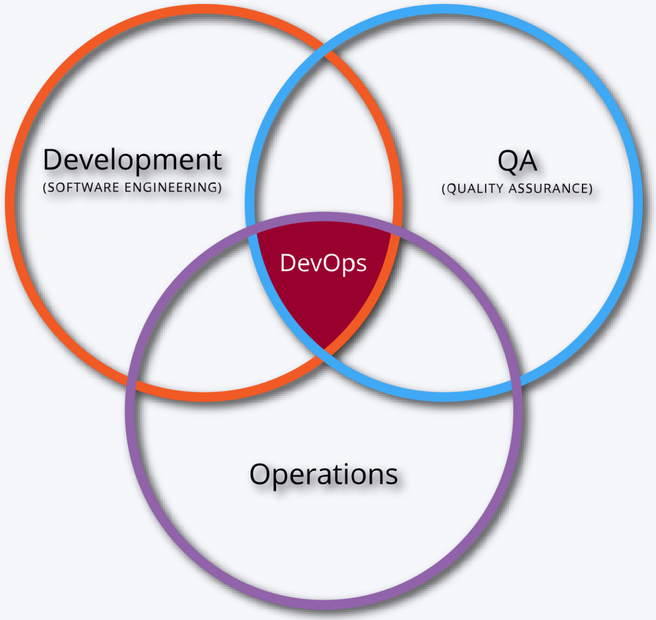
In uno scenario DevOps, il processo di sviluppo e test QA è integrato e adotta un approccio collaborativo.

In DevOps, il QA alimenta rapidamente nuovo codice di sviluppo nell'ambiente di produzione con la collaborazione del team DevOps, che quindi fornisce gli strumenti e l'infrastruttura necessari per renderlo un processo fluido, assicurandosi che tutte le modifiche funzionino come previsto. Al fine di garantire la qualità durante tutto il ciclo di consegna, entrambi i team (QA e sviluppo) condividono la responsabilità.

Pertanto, tutti coloro che sono coinvolti in DevOps sono responsabili della qualità del prodotto e di una consegna tempestiva.

Qual’è il ruolo del QA in DevOps?

Per il QA, i seguenti punti devono essere rispettati in un ciclo DevOps:



- Tutto il lavoro deve essere allineato agli standard previsti dalla procedura
- Casi di test automatizzati con massima copertura del codice
- L'installazione dovrebbe essere automatizzata
- I casi di test automatizzati devono essere adeguatamente allineati con il ciclo di integrazione continua

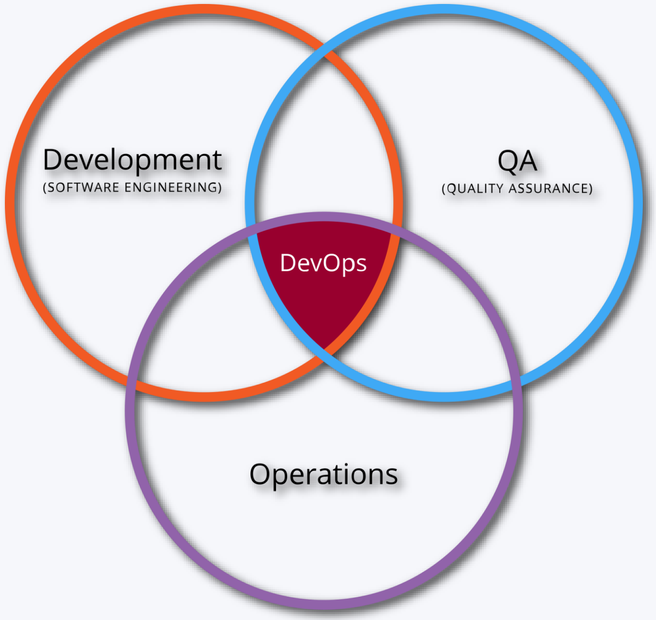
12

Responsabilità QA in DevOps

Responsabilità QA in DevOps

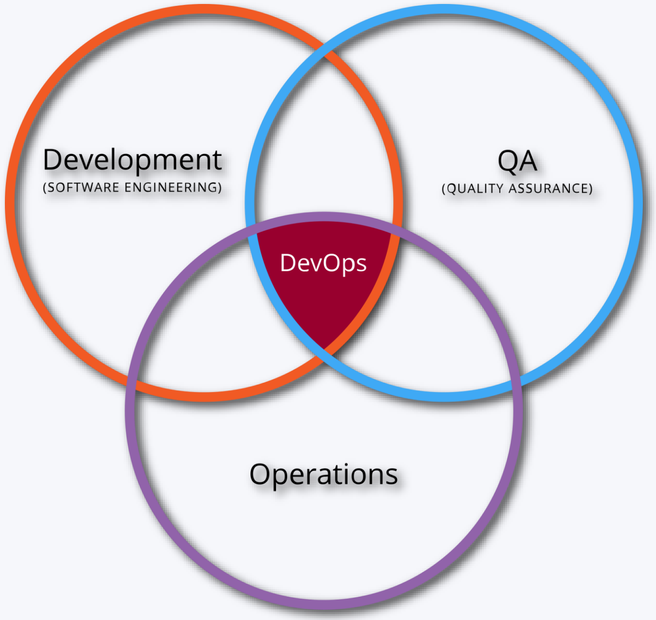
QA ha una serie di responsabilità in DevOps: di seguito facciamo luce su alcuni di loro:

- Il QA dovrebbe mirare a rilevare un bug nel primo punto del ciclo e anche a prevenire la ricomparsa di potenziali bug nell'ambiente di produzione
- Il QA non dovrebbe limitare il proprio ruolo a trovare e prevenire i difetti, ma dovrebbe anche evidenziare i problemi nei processi e raccomandare modifiche laddove necessario. Questa pratica è possibile solo se il QA assume la proprietà del processo di miglioramento in DevOps
- Il QA dovrebbe assicurarsi che tutti gli ambienti necessari per i test siano standardizzati e le distribuzioni automatizzate. Non c'è spazio per i test manuali
- Essendo un sostenitore della qualità, il QA ha un'influenza sia sullo sviluppo che sulle operazioni. Non dovrebbero solo trovare difetti, ma anche notare qualsiasi opportunità per migliorare la qualità del prodotto. In questo modo, la qualità sarà controllata in ogni fase.



Responsabilità QA in DevOps

I processi si stanno muovendo verso test più rapidi e creazione di un'implementazione al fine di soddisfare le esigenze sempre crescenti dei clienti, quindi tenete presente che il QA aiuta a soddisfare e raggiungere tutti gli obiettivi in DevOps.



Questa situazione ha creato la necessità di un controllo costante della qualità per fornire prodotti impeccabili.

13

Il ruolo strategico del QA in DevOps

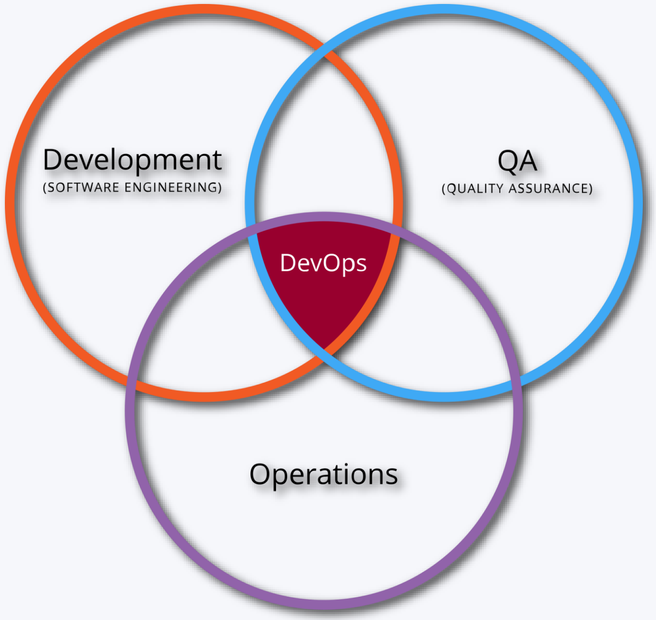
Il ruolo strategico del QA in DevOps

Nell'approccio DevOps, i tester e gli sviluppatori svolgono ruoli altrettanto importanti.

I confini delle responsabilità lavorative tra uno sviluppatore e un tester diventano sfocati in DevOps.

QA lega insieme lo sviluppo e le operazioni e consente loro di collaborare per avere software e applicazioni operativi.

Tutti nell'organizzazione si assumono la responsabilità della qualità e della stabilità, e quindi del successo aziendale.

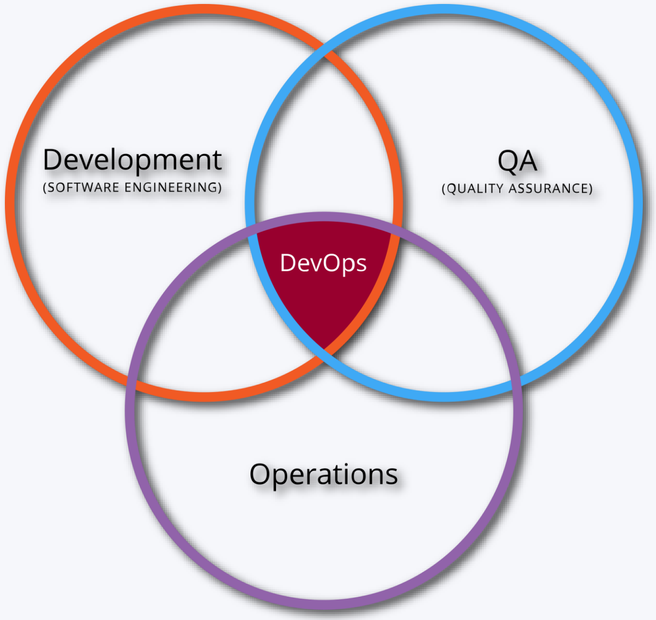


Il ruolo strategico del QA in DevOps

Uno sviluppatore pensa più come un tester e controlla continuamente il proprio codice per potenziali errori.

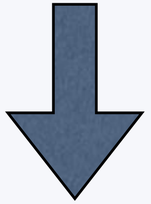
Al contrario, un tester, la cui responsabilità è quella di testare l'applicazione/software utilizzando diversi tipi di strumenti e metodologie di test, corregge anche il codice.

In un tale scenario, le organizzazioni senza una strategia di QA su test forti e definiti per "testare continuamente" in DevOps affrontano problemi nel raggiungere i loro obiettivi anche e sopratutto per il poco tempo a disposizione e per velocizzare la consegna del prodotto finito

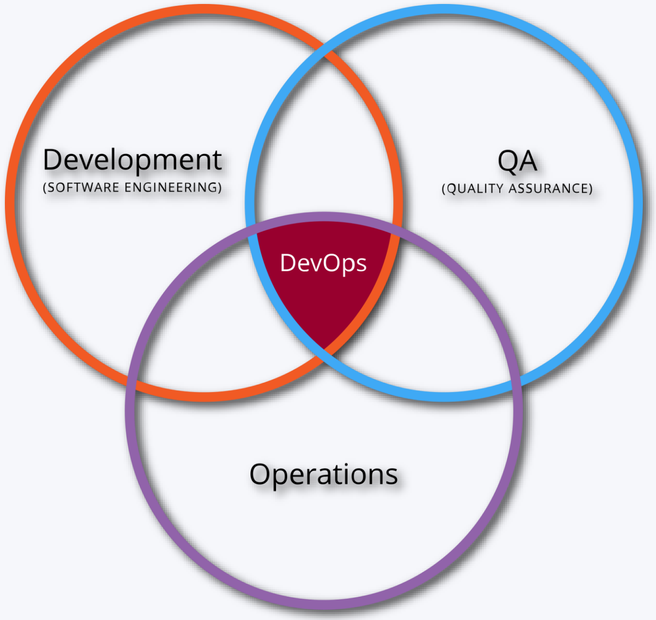


Il ruolo strategico del QA in DevOps

I test continui, se implementati diligentemente, consentono una continuità impeccabile nel ciclo di consegna.



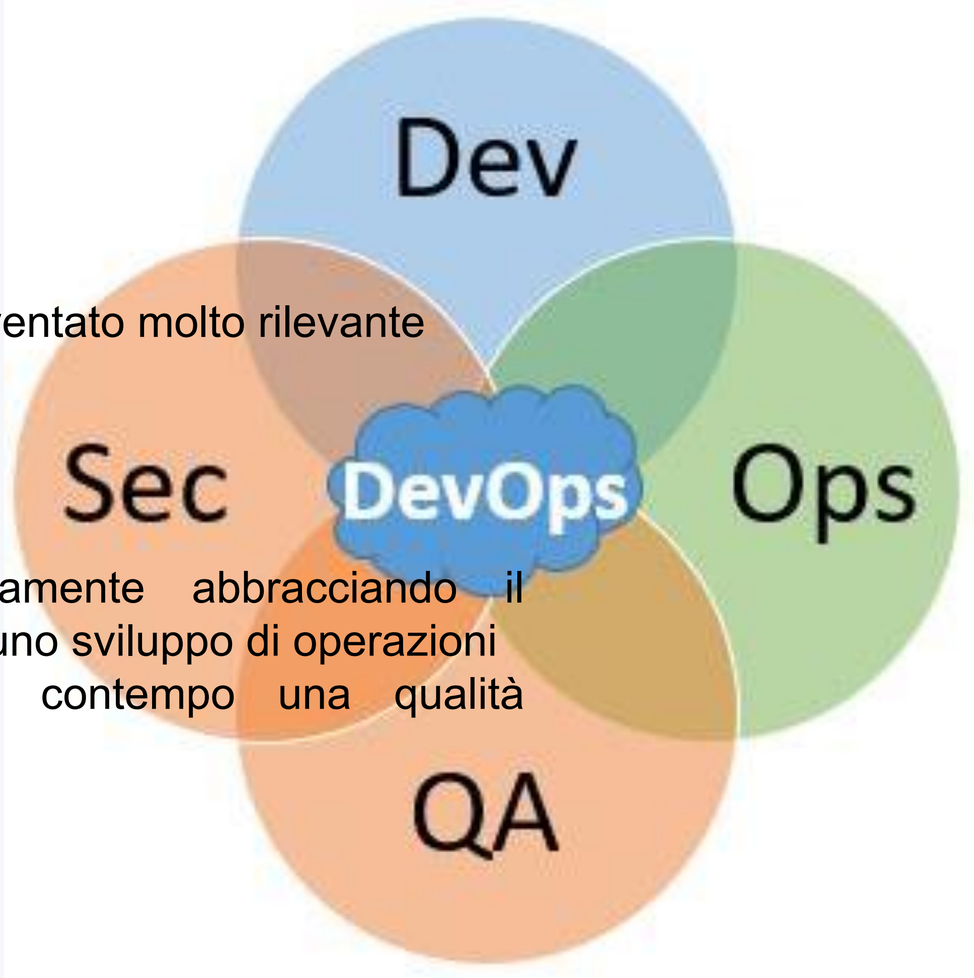
Riduce il tempo dallo sviluppo alle operazioni

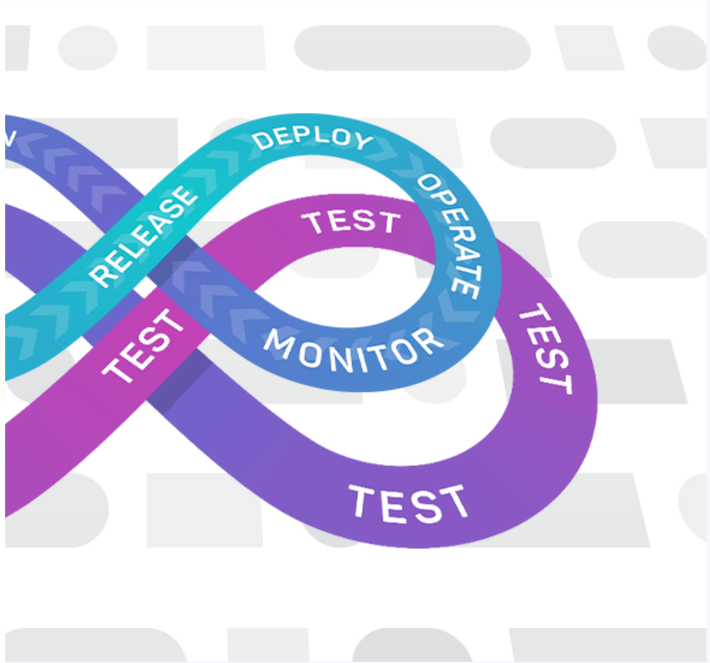


Il QA sta entrando in un ruolo più strategico di costruzione di una solida infrastruttura di test del software, monitorando la qualità in tutto l'SDC (Software Development Cycle)

Il ruolo strategico del QA in DevOps

Avere una strategia QA in atto, è diventato molto rilevante





Le organizzazioni stanno prontamente abbracciando il movimento DevOps per consentire uno sviluppo di operazioni Software efficienti, ottenendo al contempo una qualità superiore

14

10 Best Practies QA in uno scenario DevOps

10 Best Practies QA in uno scenario DevOps

1

I team di QA e test dovrebbero far parte dei team tecnici.

Devono andare oltre i test funzionali manuali e dovrebbero concentrare i loro sforzi sull'automazione e sulle strategie di test. Il QA diventa il fattore abilitante della qualità in tutta SDLC (Software Development Life Cycle)

2

La qualità deve essere ben definita per soddisfare i requisiti.

Piuttosto che cercare di raggiungere la perfezione nel software, il QA dovrebbe muoversi per concentrarsi sul raggiungimento di un'esperienza utente soddisfacente entro il periodo di tempo specificato.

3

Le metriche che misurano la qualità devono essere stabilite e misurate.

Tali metriche dovrebbero rilevare i difetti del software all'inizio del ciclo di sviluppo.

10 Best Practies QA in uno scenario DevOps

4

Gli obiettivi degli individui e dei team devono essere ottimizzati.

Le organizzazioni devono rafforzare il comportamento necessario e il cambiamento culturale incentivando la garanzia della qualità.

5

I requisiti devono essere specifici.

I team di QA dovrebbero coinvolgere in modo proattivo il processo dei requisiti per aiutare e guidare i team di sviluppo verso la giusta direzione.

6

Il controllo qualità e i test dovrebbero concentrarsi maggiormente sui test di regressione automatizzati su aree critiche come le funzionalità chiave del software.

10 Best Practies QA in uno scenario DevOps

7

QA e test deve muoversi verso lo sfruttamento degli strumenti di automazione per automatizzare i test ove possibile.

8

I team di sviluppo, operazioni e QA dovrebbero essere facilitati e incoraggiati a comunicare, collaborare e ottimizzare i loro sforzi.

9

L'integrazione continua è la chiave per identificare i difetti all'inizio del ciclo di vita dello sviluppo.

Affinché l'integrazione continua sia attuata in modo efficace, tutte le parti interessate devono integrare il loro lavoro spesso su base giornaliera.

10

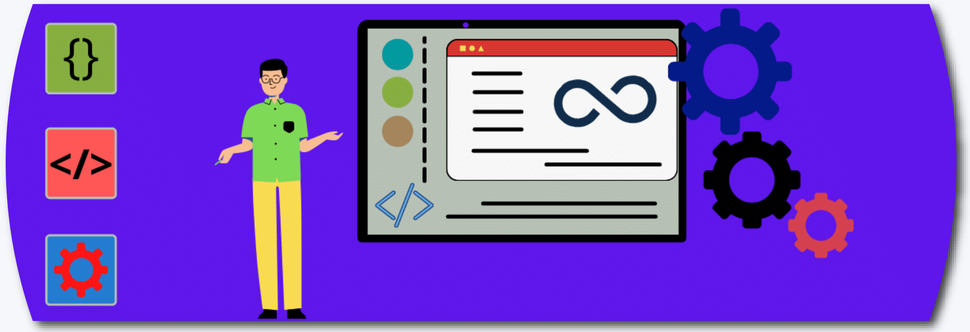
Un processo di test continuo completamente automatizzato deve essere integrato nell’SDCC (Software Designer Data Center) per un processo di consegna continua di successo che riduca al minimo i rischi, riduce i costi e accelera il time-to-market con rilasci frequenti.

15

Strumenti DevOps: quali sono i migliori Tool DevOps da conoscere nel 2023

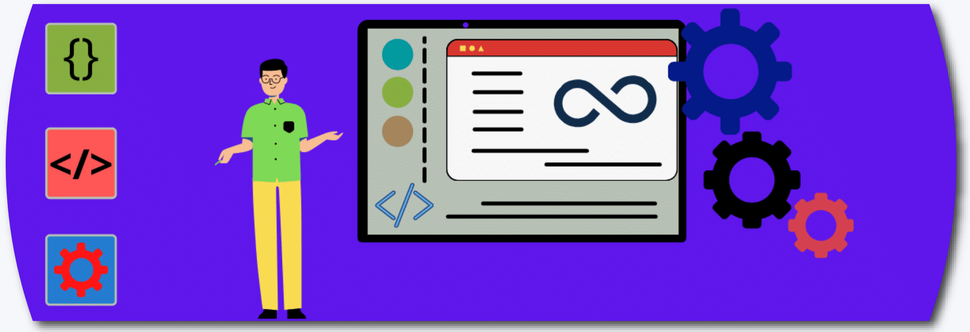
Strumenti DevOps: quali sono i migliori Tool DevOps da conoscere

Quali sono i migliori tool DevOps?


{}
</></</></</></></>

Strumenti DevOps: quali sono i migliori Tool DevOps da conoscere nel 2023

- Strumenti DevOps – Versioning
- Tool DevOps – Monitoring
- Strumenti DevOps – CI/CD e gestione della configurazione
- Strumenti DevOps – gestione container
- DevOps tool – comunicazione
- Altri strumenti DevOps



Strumenti DevOps – Versioning

GitHub

GitHub è uno degli strumenti di gestione di repository Git più usati in ambito DevOps.

Con questo tool infatti i developer possono lavorare in modo coordinato sulla stessa base, pur sviluppando in modo indipendente il codice. Fanno ‘concurrent DevOps’, come si dice. Ogni modifica viene notificata a tutti i collaboratori, che vengono quindi aggiornati in tempo reale sull’evoluzione del progetto. Inoltre è possibile ripercorrere l’intera storia del codice attraverso lo storico dei commit – fondamentale per correggere in pochi secondo un deploy sbagliato.

Bitbucket

Anche Bitbucket è uno strumento che permette di gestire il codice di un progetto attraverso l’intero ciclo di sviluppo software, come GitHub.
Nelle funzionalità principali i due tool DevOps si assomigliano: i vantaggi riconosciuti a Bitbucket sono la possibilità di gestire gratuitamente repository privati (anche su GitHub è possibile farlo, ma solo a pagamento), e il grande ecosistema di strumenti integrati, a partire da Jira e Trello, che aiutano nella gestione complessiva del progetto.

Tool DevOps – Monitoring

Nagios

Nagios è uno strumento DevOps particolarmente utile per il monitoraggio di computer e risorse di rete.

Infatti monitora il funzionamento di reti, infrastrutture e servizi, creando un sistema di alert che consente di intervenire tempestivamente ad ogni segnale di malfunzionamento. Le funzioni di reportistica e i grafici di performance permettono poi di elaborare trend utili a prevenire futuri problemi.

Sentry

Sentry è uno dei migliori strumenti DevOps per il rilevamento di errori o bug, fornito di un ampio supporto per la maggior parte dei linguaggi di programmazione e dei framework.

Questo tool scansiona le linee del codice alla ricerca di ogni segnale che possa indicare la presenza di un bug. Invia poi report e notifiche ai programmatori che mettono in evidenza il tipo di errore rilevato, la sua localizzazione all’interno del codice e una scelta di possibili soluzioni per risolverlo.

Strumenti DevOps – CI/CD e gestione della configurazione

Jenkins

Jenkins è un server open source di CI/CD (Continuous Integration, integrazione continua, e Continuous Deployment, implementazione continua) scritto in Java.

Consente di automatizzare le diverse fasi del ciclo di vita del software, dalla creazione alla distribuzione. Infatti grazie a Jenkins i programmatori possono ad esempio fare automaticamente il commit del proprio codice all'interno di una repository, lanciare test ed analizzarne i risultati, per avere un feedback immediato sulla sua qualità.

Ansible

Ansible è un server open source di gestione della configurazione e può essere utilizzato su qualsiasi tipo di infrastruttura a prescindere dalle tecnologie usate.

Consente di automatizzare processi come la configurazione di rete, la creazione di ambienti di sviluppo e l'implementazione cloud. Ansible permette quindi di replicare in modo automatico molte delle procedure manuali di configurazione impostate dai programmatori: in questo modo vengono abbassati i costi dell'infrastruttura e viene aumentata la scalabilità dei sistemi informatici.

Strumenti DevOps – CI/CD e gestione della configurazione

Puppet

Puppet è uno strumento DevOps open source di configuration management del software.

Consente di automatizzare l'intero ciclo di sviluppo software, dal provisioning alla configurazione, dal testing al reporting. Puppet include un proprio linguaggio dichiarativo, usato per descrivere la configurazione del sistema.

Chef

Chef è un tool di gestione della configurazione del software, che utilizza un linguaggio imperativo.

Questo consente agli sviluppatori di programmare ogni aspetto del proprio sistema, esponendosi però al rischio di una eccessiva complicazione delle procedure.

Strumenti DevOps – gestione container

Docker

Docker è una piattaforma open source per lo sviluppo, la distribuzione e l'esecuzione di applicazioni.

Basato su Linux, Docker consente di separare le applicazioni dall'infrastruttura: infatti permette di immagazzinare ed eseguire un'applicazione in un ambiente isolato chiamato container. I container contengono al proprio interno tutto ciò che è necessario per eseguire l'applicazione, come codice sorgente, file di supporto, runtime e file di configurazione del sistema. In questo modo puoi essere indipendente dall'infrastruttura del sistema che ospita l'esecuzione dell'applicazione – con un notevole risparmio di tempi e costi di sviluppo, controllo e distribuzione. Inutile dire che anche la containerizzazione è tra le componenti fondamentali dei processi DevOps.

Kubernetes

Kubernetes è una piattaforma open source per l'automazione delle operazioni sui container Linux: permette di creare e gestire con facilità sistemi distribuiti basati sull'architettura a container.
Inoltre Kubernetes supporta soluzioni di hosting trasversali, su cloud pubblici, privati o ibridi, consentendo di orchestrare la configurazione e la gestione automatica di sistemi informatici, applicazioni e servizi spostando i carichi di lavoro dall'una all'altra risorsa a seconda delle esigenze. Suddividere applicazioni di grandi dimensioni in microservizi eseguiti su container può favorire notevolmente la velocità di sviluppo e l'affidabilità del software prodotto.

DevOps tool – comunicazione

Slack

Slack è uno degli strumenti più efficaci per collaborare e comunicare in tempo reale.

Organizzazioni e aziende di tutto il mondo utilizzano Slack per rimuovere le barriere comunicative, e offrire ai membri del team uno spazio condiviso in cui lavorare. Inoltre il grande numero di di integrazioni possibili con altri sistemi e piattaforme consente di centralizzare la gestione del progetto. In questo modo gli sviluppatori hanno un unico ambiente per collaborare tra di loro e comunicare con tutti gli altri membri del team.

Microsoft Teams

Microsoft Teams, come Slack consente di creare un ambiente condiviso di comunicazione tra tutti i professionisti coinvolti in un progetto di sviluppo.
Inoltre è possibile integrare altri strumenti all'interno di MS Teams, ad esempio per ricevere alert su incidenti di produzione e errori di distribuzione direttamente nello spazio condiviso. MS Teams è particolarmente indicato per le realtà che lavorano con altri strumenti della suite Microsoft, grazie al maggiori possibilità di collaborazione rese possibili dalle integrazioni native.

Altri strumenti DevOps

Vagrant

Vagrant è il tool che ha risolto l’annoso problema del ‘eppure sulla mia macchina funziona’.

Nessuno potrà più usare quella scusa, perché Vagrant permette di creare e gestire macchine virtuali e ambienti di esecuzione condivisi. Inoltre assicura che l’ambiente rimanga immutato su ogni dispositivo, permettendo di lavorare in modo organico e condiviso.

Splunk Phantom

La sicurezza informatica del software è un elemento di primaria importanza per un DevOps, ed è proprio sotto questo aspetto che Splunk Phantom si dimostra un tool estremamente utile.

Consente infatti di costruire un’infrastruttura con elevata capacità difensiva già dalle prime fasi del processo di sviluppo del software. Phantom permette infatti a tutti i DevOps di collaborare alla limitazione dei rischi di cyber security all’interno di un ambiente centralizzato.

Altri strumenti DevOps

Elastic

In realtà Elastic non è il nome di un tool, ma dell’organizzazione che gestisce i diversi tool che formano uno stack estremamente utile per ogni DevOps: lo stack ELK.

L’Elastic stack è composto da tre tool diversi, Elasticsearch, Logstash e Kibana. Usando questi strumenti è possibile analizzare ed elaborare log e dati strutturati e non grazie a Logstash, eseguirne l’indicizzazione e la ricerca con Elasticsearch, per poi visualizzarli e filtrarli in tempo reale tramite Kibana. Il tutto è poi potenziabile grazie alle numerose estensioni, che forniscono funzionalità di monitoraggio, reporting e alert, machine learning, graph analytics e sicurezza informatica.

16

Conclusioni

Conclusioni

Il ruolo del QA sta cambiando il clima attuale, dove dobbiamo garantire la consegna tempestiva dei prodotti concentrandoci maggiormente sulle esigenze del cliente.

Un pericolo intrinseco all'interno della crescente domanda di velocità è il rilascio della build in produzione senza prestare sufficiente attenzione alla garanzia della qualità in tutti i punti del ciclo di vita dell'applicazione.

Compromessi come questo possono essere a dir poco un disastro!!!!

Conclusioni

Quando il modello “Agile” apparve per la prima volta, presto crebbe in popolarità superando infine il modello tradizionale per diventare la scelta preferita per lo sviluppo del software.

Il futuro appartiene a DevOps.

Si tratta di un processo continuo che continua a portare miglioramenti. In scenari faticosi e impegnativi, c'è una costante necessità di testare il prodotto insieme allo sviluppo per garantire tempi di commercializzazione più rapidi mantenendo in vista anche l'efficacia in termini di costi.

Come abbiamo visto insieme, tutto ciò che è stato discusso approva il ruolo del QA con DevOps, rendendoli inseparabili.

Conclusioni

Il QA è parte integrante del processo DevOps e, portando il meglio di entrambi i mondi, possiamo realizzare un prodotto di qualità in tempo utile